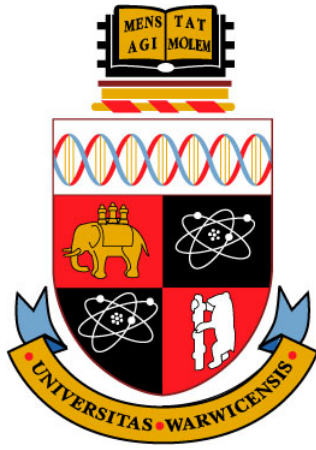




# Counting Complexity of PureCircuit -1

CS907 Dissertation Project

**Christos Demetriou 2018918**

Supervisor: Dr. Christian Ikenmeyer

**Department of Computer Science**

University of Warwick

2024-2025

---

## Abstract

When this thing gets up to 88 mph, you're gonna see some serious s...

*Keywords: Flux Capacitor, 1.21 Gigawatts, Calvin Klein*

---

## Acknowledgements

Always good to acknowledge people.

---

# Abbreviations

|                                  |     |
|----------------------------------|-----|
| Turing Machine                   | TM  |
| Non-Deterministic Turing Machine | NTM |

# Contents

|                                                                       |           |
|-----------------------------------------------------------------------|-----------|
| Abstract . . . . .                                                    | ii        |
| Acknowledgements . . . . .                                            | iii       |
| Abbreviations . . . . .                                               | iv        |
| List of Figures . . . . .                                             | viii      |
| List of Tables . . . . .                                              | ix        |
| List of Algorithms . . . . .                                          | x         |
| List of Notations . . . . .                                           | xi        |
| <b>1 Introduction</b>                                                 | <b>1</b>  |
| 1.1 Introduction . . . . .                                            | 2         |
| 1.2 Theoretical Preliminaries . . . . .                               | 4         |
| 1.2.1 Search problems . . . . .                                       | 4         |
| 1.2.2 Counting Complexity and Combinatorial Interpretations . . . . . | 13        |
| 1.2.3 Kleene Logic and Hazard-Free Circuits . . . . .                 | 15        |
| 1.3 Literature Review . . . . .                                       | 18        |
| 1.3.1 Project Question and Objectives . . . . .                       | 19        |
| <b>2 Main findings</b>                                                | <b>21</b> |
| 2.1 Theory Analysis . . . . .                                         | 22        |
| 2.1.1 PureCircuit and EndOfLine . . . . .                             | 22        |
| 2.1.2 SourceOrExcess analysis . . . . .                               | 34        |
| 2.1.3 PPAD Counting Hierarchies . . . . .                             | 41        |
| 2.1.4 Supplementary Findings . . . . .                                | 43        |
| 2.2 Software Deliverable . . . . .                                    | 46        |
| 2.2.1 Tooling Selection . . . . .                                     | 46        |
| 2.2.2 Program Visualisation Summary . . . . .                         | 47        |
| 2.2.3 Solution Selection . . . . .                                    | 47        |
| 2.2.4 Solution Enumeration . . . . .                                  | 48        |
| 2.2.5 Future Extensions . . . . .                                     | 51        |

---

|                                 |           |
|---------------------------------|-----------|
| <b>Appendices</b>               | <b>55</b> |
| Appendices . . . . .            | 56        |
| .1    The Flux Sketch . . . . . | 56        |

# List of Figures

|                                                                                                                                                                                                                                                                                                                   |    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.2.1 Types of subgraphs in ENDOfLINE . . . . .                                                                                                                                                                                                                                                                   | 5  |
| 1.2.2 Figure redrawn from [7] . . . . .                                                                                                                                                                                                                                                                           | 7  |
| 1.2.3 Edge between points $i, j$ of the original graph. Main observation is the fact that each point comes with a pair of exponentially long lines, which we refer to as the input output lines. . . . .                                                                                                          | 7  |
| 1.2.4 $K_3$ graph construction. Combinations of colours indicate an edge between the two nodes. Solid dots indicate edges from 0, crossed lines indicate edges from 1 and stripped lines indicate edges from 2. We Add two coloured stripped circles on the 4 degree nodes to indicate the $E_n^2$ edges. . . . . | 8  |
| 1.2.5 Depiction of the edge removal. Main idea is that this we keep the number of sources or sinks as well as invertible (we can trace the source or sink from the original instance), despite losing a lot of graph information. . . . .                                                                         | 8  |
| 1.2.6 A colouring example of a subdivided simplex. As we can observe, no matter how we modify colourings, there will always be a small simplex that contains all colours                                                                                                                                          | 10 |
| 1.2.7 Visual interpretation of the <i>StrongSperner</i> problem. Figure from paper [12] . . . . .                                                                                                                                                                                                                 | 11 |
| 1.2.8 Example on an $8 \times 8$ chessboard with a missing square. As we can observe, there will always be one square we cannot tile . . . . .                                                                                                                                                                    | 13 |
| 1.2.9 Example of reducing a 6x6 sudoku puzzle, into a graph colouring problem . . . . .                                                                                                                                                                                                                           | 14 |
| 1.2.10 Computation tree of an ATM with labellings . . . . .                                                                                                                                                                                                                                                       | 18 |
| 2.1.1 . . . . .                                                                                                                                                                                                                                                                                                   | 22 |
| 2.1.2 Dimension doubling. We demonstrate the example in two dimensions. . . . .                                                                                                                                                                                                                                   | 23 |
| 2.1.3 Overlapping Sperner Solutions . . . . .                                                                                                                                                                                                                                                                     | 25 |
| 2.1.4 Pure Circuit construction visualisation . . . . .                                                                                                                                                                                                                                                           | 27 |
| 2.1.5 Purification gadget. The red nodes denote the outputs of the purification gadget. . . . .                                                                                                                                                                                                                   | 27 |
| 2.1.6 Shielding method on 2D-EoL embeddings. . . . .                                                                                                                                                                                                                                                              | 29 |
| 2.1.7 2D EndOfLine Bipolar Colouring . . . . .                                                                                                                                                                                                                                                                    | 30 |
| 2.1.8 Transformed domain mapping example. . . . .                                                                                                                                                                                                                                                                 | 31 |
| 2.1.9 Snake Embedding technique between the folding dimension and the new dimension                                                                                                                                                                                                                               | 32 |
| 2.1.10 Snake Embedding technique between the folding dimension and the new dimension                                                                                                                                                                                                                              | 33 |

---

|        |                                                                                                                                                                                                         |    |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1.11 | Panchromatic edge for 2 dimensions . . . . .                                                                                                                                                            | 35 |
| 2.1.12 | The current table shows how we deal with excess nodes. All other nodes such as sources, sinks, lines or isolated vertices stay the same. . . . .                                                        | 36 |
| 2.1.13 | Colouring example based on the proposed circuit . . . . .                                                                                                                                               | 39 |
| 2.1.14 | Example colouring where the arrow denotes the direction. In between the red and blue cubes in the bottom corners we have the positive line. The brown cubes around denote the negative labels . . . . . | 40 |
| 2.1.15 | Colouring around the double sink points. The colours correspond to the colour legend of 2.1.13 . . . . .                                                                                                | 40 |
| 2.1.16 | TARSKI construction . . . . .                                                                                                                                                                           | 43 |
| 2.1.17 | Equal solution conversion . . . . .                                                                                                                                                                     | 45 |
| 2.1.18 | Not equal solution . . . . .                                                                                                                                                                            | 45 |
| 2.2.1  | ECS visualisation implementation . . . . .                                                                                                                                                              | 46 |



# List of Tables

|                                          |    |
|------------------------------------------|----|
| 1.2.1 Three-valued logic [26]            | 9  |
| 2.1.1 Kleene Unary Examples              | 24 |
| 2.2.1 <b>AND</b> value propagation rules | 50 |
| 2.2.2 <b>OR</b> value propagation rules  | 50 |

# List of Algorithms

2.1.1 Turing machine  $F$  with input  $(p_1, p_2) \in B_{2^{4k+10}}$  . . . . . 30

2.2.1 PureCircuit Backtracking Algorithm Enumerator . . . . . 48

---

# List of Symbols

$[n]$  List of numbers  $1, \dots, n$ .

$[n]_0$  Defined as  $[n] \cup \{0\}$ .

$\mathbb{N}$  Set of natural numbers excluding 0.

$\mathbb{N}_0$  Natural numbers including 0.

$n^{O(1)}$  Set of all functions  $f : \mathbb{N} \rightarrow \mathbb{N}$ , where  $f \in O(n^c)$  for some  $c \in \mathbb{N}$ .

$x_{-i}$  Given  $x \in A^d$ , we use  $x_{-i}$  to denote all members excluding index  $i$ .

$x_{-i}(e) \equiv x(x_{-i}, e)$  Given  $x \in A^d$ , we denote  $x_{-i}(e)$  or  $x(x_{-i}, e)$  to indicate  $n$  dimensional vector, where

$$\forall j \in [n] : [x_{-i}(e)]_j = [x(x_{-i}, e)]_j = \begin{cases} e & \text{if } j = i \\ x_i & \text{otherwise} \end{cases}$$

Will be used to indicate element replacement.

$[x]_j$  Given  $x \in A^d$  for some  $j \in [d]$ , we have  $[x]_j = x_j$ .

$\mathbb{B}$  The binary set  $\{0, 1\}$ .

$\mathbb{T}$  Defines the set  $\{0, 1, \perp\}$  which depicts the kleene algebra.

# **Chapter 1**

## **Introduction**

---

## 1.1 Introduction

Combinatorics is a field of study in mathematics that primarily focuses on the notion of counting objects with certain properties. Over time, this notion has shifted, especially in the subfield of algebraic combinatorics, where there is no clear notion of the object that we are counting, and the numbers express something more abstract [30]. Researchers have started asking whether it is possible to ask the inverse question, or more informally, can we associate our sequence of numbers to sets of objects? [23, 30, 24]

To understand this notion, we provide an example from the papers by Ikenmeyer and Pak [30, 23]: Assume that  $f(G)$  counts the number of 3-colourings of a graph  $G$ . Now one can, ask the question: does the function  $h(G) \triangleq f(G)/6$  also count something? The answer is yes as argued in [30, 23] and the idea is, as for every colouring  $\chi$ , we can compute  $3!$  additional colourings since we can permute the colours we assigned. Therefore we only need to count one of the permutations, or the smallest colouring. We can therefore argue that  $h(G)$  counts the smallest possible 3-colourings of a graph  $G$ . We say that this is a “combinatorial interpretation of  $h(G)$ ”.

But not all non-negative functions have such a clear interpretation. We use an example by Ikenmeyer et al. [23]. We recall a combinatorial variant of the *Pigeonhole principle*, where given a circuit  $C : \{0,1\}^n \rightarrow \{0,1\}$ , find  $x$  such that  $C(x) = 0^n$  or distinct  $x, y \in \{0,1\}^n$  such that  $C(x) = C(y)$  and  $C(C(x)) \neq 0$ .  $C$  essentially maps  $\{0,1\}^n \rightarrow \{0,1\}^n \setminus \{0^n\}$ . Every point that is mapped to  $0^n$  is considered a violation. The problem counts every pair of points that is mapped to the same bin or to a point that is a violation. We add the restriction of  $C(C(x)) \neq 0$  to avoid double counting. We can clearly observe that, if  $e(C)$  counts the number of solutions, under the pigeonhole principle, it must be the case  $e(C) \geq 1$  since we either have a violation or there exists at least one pair of points that is mapped to the same bin by the pigeonhole principle. So now we can ask the question: does the function  $e'(C) \triangleq e(C) - 1$  also have a set of object which we can enumerated over? Ikenmeyer et al. [23] demonstrated that this problem does not have a combinatorial interpretation. This leads to the questions of how can we formally represent the meaning of combinatorial interpretations and what can we do to verify whether a function has one?

In recent years, there has been a revolutionary initiative to answer the aforementioned questions, with the help of counting complexity theory and the complexity class of  $\#P$ . In fact, being able find such definitions or interpretations is crucial, as it allows us to utilise tools from enumerative combinatorics to understand and reveal hidden structures and properties of such functions [30, 23]. Moreover, there are several problems or numbers, such as *Kronecker coefficients* [28], whose combinatorial interpretation could lead to groundbreaking breakthroughs, such as a step closer to the resolution of the  $P \neq NP$  conjecture [23, 22].

In our current work, we focus on extending the work done by Ikenmeyer et al. [23], where they focused on the creation of frameworks that determine whether  $f \in^? \#P$ , by looking at the subclasses of  $\#TFNP - 1$  problems, a class of problems whose combinatorial nature guarantee the existence of a solution. More specifically, we look into the **PPAD** class which seems to have several problems where under decrementation, may or may not have a combinatorial interpretation [23]. This implies that **PPAD** acts as some sort of barrier, to  $\#P$  which motivates the research as to what causes

To accomplish this task, we studied *PureCircuit* which was created by Deligkas et al. [15], as we hope its **PPAD**-completeness and circuit-like nature may give us useful insights to the counting complexity of **PPAD** - 1.

We will now give an extensive introduction to the literature and the related problems and then offer formal and precise objectives that capture our goal. We will then go over the main results of our theory research with the conjectures and possible directions that we uncovered. Moreover,

---

we will give an overview of the visualisation software we developed to verify our theoretical results and give an overview of its capabilities and limitations. Lastly we will give brief analysis of our project management, the lessons we learnt and a conclusion that summarises our project as a whole.

## 1.2 Theoretical Preliminaries

Our work lies in the intersection of the three core fields: counting complexity, total search problems and Kleene logic.

### 1.2.1 Search problems

We offer brief preliminaries to function problems and the different kinds that exist. First and foremost when

#### Definition 1.2.1: Search Problems

**Search problems** can be defined as relations  $R \subseteq \mathbb{B}^* \times \mathbb{B}^*$ , where given  $x \in \mathbb{B}^*$ , we want to find  $y \in \mathbb{B}^*$  such that  $xRy$ .

**Total Search problems** are search problems such that for each input, there must exist at least one solution.

Like decision problems, we can define their search analogues using the definitions in 1.2.1. Moreover, we use the *Levin reductions* 1.2.1 to associate two problems.

#### Definition 1.2.2: FNP and TFNP

**FNP** are *search problems* such that there exists poly-time TM  $M : \mathbb{B}^* \rightarrow \mathbb{B}$  and a poly function  $p : \mathbb{N} \rightarrow \mathbb{N}$  such that:

$$\forall x \in \mathbb{B}^*, y \in \mathbb{B}^{p(|x|)} : xRy \iff M(x, y) = 1$$

Lastly **TFNP** =  $\{L \in \mathbf{FNP} \mid L \text{ is total}\}$

#### Definition 1.2.3: Levin Reductions

Given two search problems  $R, R'$ , a pair of poly-time computable functions  $f, g$  where  $f, g : \{0, 1\}^* \rightarrow \{0, 1\}^*$ , is called a *Levin reduction*, if

$$\begin{aligned} S_R &\triangleq \{x \mid \exists y : xRy\} \\ R(x) &\triangleq \{y \in \{0, 1\}^* \mid xRy\} \\ \forall x \in \{0, 1\}^* : x \in S_R &\implies f(x) \in S_{R'} \\ \forall x \in S_R, y' \in R'(f(x)) : &(x, g(x, y')) \in R \end{aligned}$$

Within the current project, we are mainly interested with the total search problems or at least a specific hierarchy, as shown in figure ???. These problems were introduced by various researchers as seen in , and all such classes the guarantee of an existence of a solution by using some combinatorial property of the nature of the problem but can be very difficult to find. In the current project we are mainly interested in **PPAD**, known as “Polynomial Parity of Augmented DiGraphs”, created by Papadimitriou [31]. We will give a formal definition of the class as well as several cornerstone problems that we will refer to in the upcoming chapters.

We will first introduce the *EndOfLine* problem 1.2.1 by Papadimitriou [31], which takes advantage of the combinatorial fact of directed graphs where a pair of odd-degree vertices come in pairs.



# Types of Subgraphs in EOL

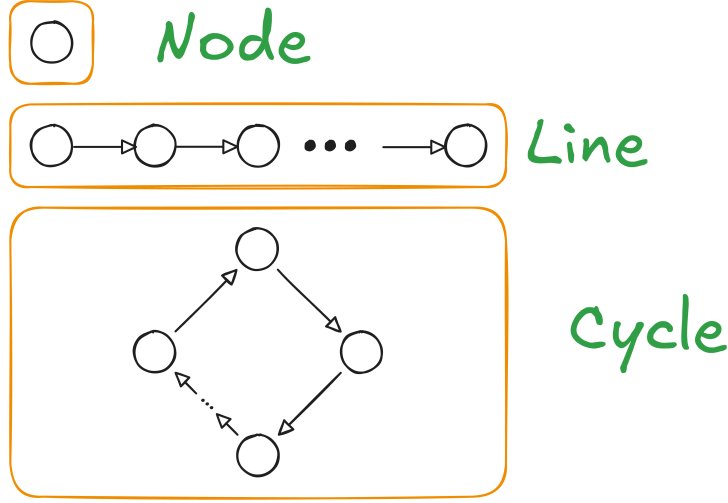


Figure 1.2.1: Types of subgraphs in ENDOFLine

## Definition 1.2.4: EndOfLine problem [31]

Given poly-sized circuits  $S, P \in \mathbb{B}^n \rightarrow \mathbb{B}^n$ , we define a digraph  $G = (V, E)$ , such that  $V = \mathbb{B}^n$  and  $E$  defined as:

$$E = \{(x, y) \in V^2 : S(x) = y \wedge P(y) = x\}$$

We define source nodes as  $v \in V : \deg(v) = (0, 1)$  and sink nodes as  $\deg(v) = (1, 0)$ . We also syntactically ensure that the  $0^n$  node is always a source, meaning  $S(P(0^n)) \neq 0 \wedge P(S(0^n)) = 0^n$ . A node  $v \in V$  is a solution iff  $\text{in-deg}(v) \neq \text{out-deg}(v)$ .

An illustrative example of an ENDOFLine instance can be seen in figure 1.2.1. From this problem, we define the PPAD complexity class 1.2.5.

## Definition 1.2.5: PPAD complexity class [31]

PPAD is defined as the set of search problems that are *levin reducible* 1.2.1 to the ENDOFLine problem 1.2.1.

### EndOfLine variants and extensions

We want introduce a small number of the several key variants of the *EndOfLine* that have been created in the past. Alexandros et al. in the paper was able to demonstrate that for several modifications of the original problem such as adding  $k$  known sources, where  $k$  can be a polynomial function with respect to the number of sources. Moreover, people have experimented with changing the degrees of the graphs, as demonstrated by Ikenmeyer et al. [23], with the problems such as SOURCEOREXCESS( $k, 1$ ) where  $k$  it the maximum in-degree for each node. All these problems remain **PPAD** complete. More interestingly, Papadimitriou et al. and Alexandros et al. were able to demonstrate that in the more general case of superpolynomial graphs, if we bound the in-degree and out-degree of each node to some polynomial of the input size, then problem remains **PPAD**. We will give the main definitions to the most relevant problems in the upcoming sections.

**Definition 1.2.6: SOURCEOREXCESS problem [23]**

We define as *SourceOrExcess*( $k, 1$ ) for  $k \in \mathbb{N}_{\geq 2}$  the search problem as such: Given a poly-sized successor circuit  $S : \mathbb{B}^n$  and a set of predecessor poly-sized circuits  $\{P_i\}_{i \in [k]}$ , we define the graph  $G = (V, E)$  such that,  $V = \mathbb{B}^n$  and  $E$  as:

$$\forall x, y \in V : (x, y) \in E \iff (S(x) = y) \wedge \bigvee_{i \in [k]} P_i(y) = x$$

We ensure that  $0^n$  is as source, meaning  $\deg(0^n) = (0, 1)$ . A valid solution is a vertex  $v$  such that  $\text{in-deg}(v) \neq \text{out-deg}(v)$ .

In addition to the aforementioned adjustments, researchers were able to give a topological interpretation to the **ENDOFFLINE 1.2.1** problem, by creating embeddings of the graph into 2D or 3D spaces as demonstrated by several researchers [7, 19, 11]. We will mainly look at two of these problems, known as **RLEAFD** and **2D-EOL** created by Chen et al. and Alexandros respectively.

**Definition 1.2.7: 2D-EOL problem [7, 19]**

**Input:** Two poly-sized circuits  $P, S : \{0, 1\}^{2n} \rightarrow \{0, 1\}^{2n}$  where  $P(0^n, 0^n) = (0^n, 0^n) \neq S(0^n, 0^n)$ .

**Output:** A point  $x \in \{0, 1\}^{2n}$  such that one of the following conditions are satisfied:

1. Sinks:  $P(S(x)) \neq x$
2. Sources:  $S(P(x)) \neq x \wedge x \neq 0^n$
3. Invalid successor:  $x - S(x) \notin \{(0, 0), (0, 1), (0, -1), (1, 0), (-1, 0)\}$
4. Invalid predecessor:  $x - P(x) \notin \{(0, 0), (0, 1), (0, -1), (1, 0), (-1, 0)\}$

The above description defines a superpolynomial such that an edge  $(x, y) \in E$  if and only if  $S(x) = y \wedge P(y) = x$  as well as the closeness constraint  $\|x - S(x)\|_1 \leq 1$ . We will now refer to a special case of 2D-EOL known as **RLEAFD** by Chen et al. where they give a planar embedding of any graph  $G$  onto a 2D plane. We find this construction useful for our results and therefore will investigate this method further.

**Planar Graph Embedding** We define the graph embedding of a graph  $G = (V, E)$  as  $G_n = (V_n, E_n)$  where  $|V| = n$  and  $V_n$  is defined as

$$\forall n \in \mathbb{N} : V_n \triangleq \left\{ \mathbf{u} \in \mathbb{N}_0^2 \mid \mathbf{u}_1 \leq 3(n^2 - 1), \mathbf{u}_2 \leq 6(n - 1) + 3 \right\}$$

$G$  can have any in- and out-degree. For a vertex  $i \in V$ , we correspond it to the node  $(0, 6i)$  in the embedded version. To define the edge set, we will first introduce the following notations.

**Definition 1.2.8:  $E(\mathbf{p}^1, \mathbf{p}^2)$**

Given two points  $\mathbf{p}^1, \mathbf{p}^2 \in V_n$  we define the set of edges  $E(\mathbf{p}^1, \mathbf{p}^2)$  as:

$$E(\mathbf{p}^1, \mathbf{p}^2) \triangleq \begin{cases} \{(\alpha, k) \mid k \in \mathbb{N} : \min\{\mathbf{p}_2^1, \mathbf{p}_2^2\} \leq k \leq \max\{\mathbf{p}_2^1, \mathbf{p}_2^2\}\} & \text{if } \mathbf{p}_1^1 = \mathbf{p}_1^2 = \alpha \\ \{(k, \alpha) \mid k \in \mathbb{N} : \min\{\mathbf{p}_1^1, \mathbf{p}_1^2\} \leq k \leq \max\{\mathbf{p}_1^1, \mathbf{p}_1^2\}\} & \text{if } \mathbf{p}_2^1 = \mathbf{p}_2^2 = \alpha \\ \emptyset & \text{otherwise} \end{cases}$$

**Definition 1.2.9:**  $E_{ij}$

Given an edge of the original graph  $(i, j) \in E$ , we define the set of edges  $E_{ij}$  as:

$$\begin{aligned} \mathbf{p}^1 &= (0, 6i) \\ \mathbf{p}^2 &= (3(ni + j), 6i) \\ \mathbf{p}^3 &= (3(ni + j), 6j + 3) \\ \mathbf{p}^4 &= (0, 6j + 3) \\ \mathbf{p}^5 &= (0, 6j) \\ E_{ij} &\triangleq E(\mathbf{p}^1 \mathbf{p}^2) \cup \dots \cup E(\mathbf{p}^4 \mathbf{p}^5) \end{aligned}$$

To formally define  $E_n$ , we first split  $E_n$  into two disjoint sets  $E_n^1$  and  $E_n^2$ . We define  $E_n^1$  as  $\bigcup_{ij \in E} E_{ij}$ . The graph generated by  $(V_n, E_n)$  has some special properties such as,  $\forall v \in V_n : \text{in-deg}(v) + \text{out-deg}(v) \leq 4$  and  $\forall v \in V_n : \text{deg}(v) = 4 \implies \text{in-deg}(v) = 2 = \text{out-deg}(v)$ . For every 4-degree vertex, we add 4 additional edges as depicted in the figure 1.2.2 depicted in blue and add them to  $E_n^2$ .

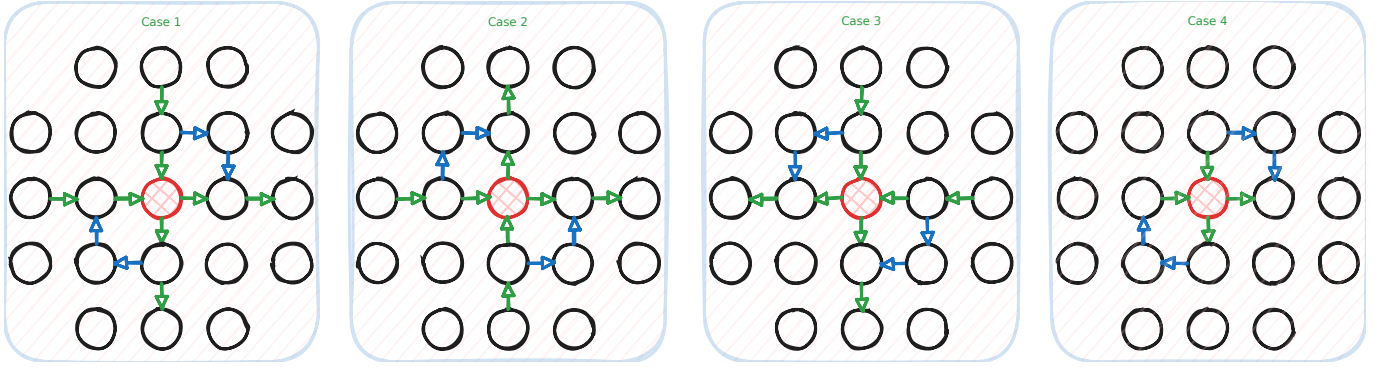


Figure 1.2.2: Figure redrawn from [7]

Given the above we have a full description of  $G_n$ . A visual explanation can be seen in figure 1.2.3 and an example of the  $K_3$  complete graph in figure 1.2.4

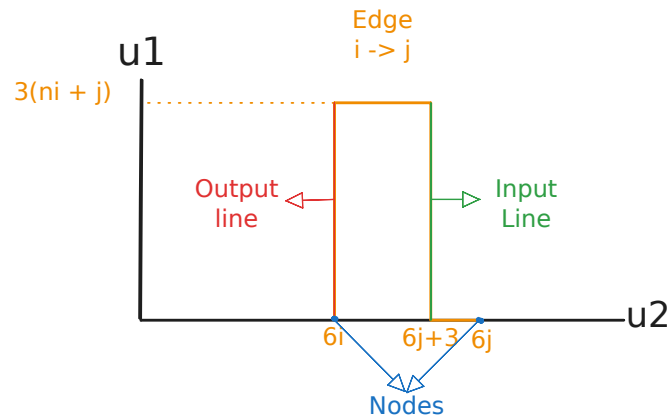


Figure 1.2.3: Edge between points  $i, j$  of the original graph. Main observation is the fact that each point comes with a pair of exponentially long lines, which we refer to as the input output lines.

**RLEAFD problem** We denote  $C_n$  as the set of planar graphs  $G_n = (V_n, E_n)$  such that  $\forall \mathbf{u} \in V_n : \text{in-deg}(v) \leq 1 \wedge \text{out-deg}(v) \leq 1$ . From that we define the class of problems RLEAFD as in definition 1.2.1.

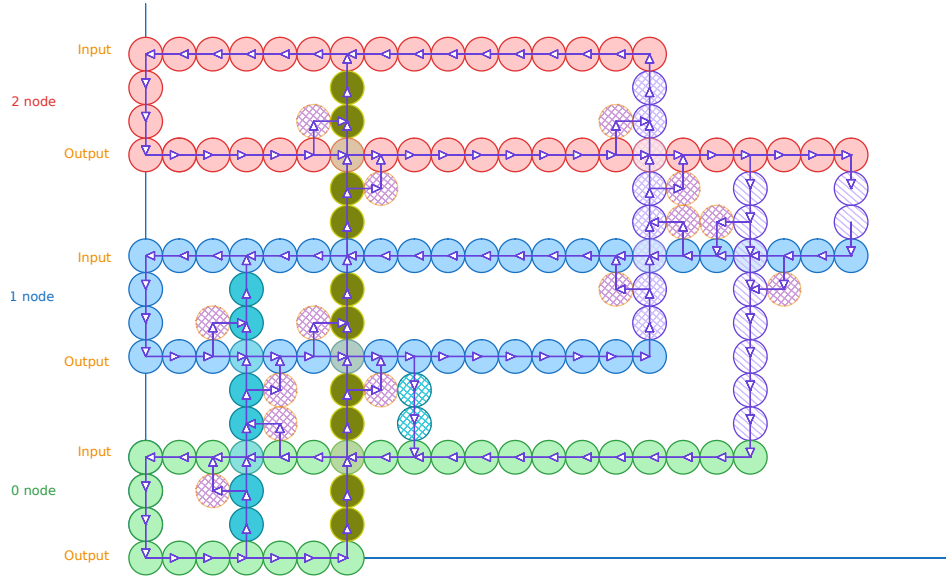


Figure 1.2.4:  $K_3$  graph construction. Combinations of colours indicate an edge between the two nodes. Solid dots indicate edges from 0, crossed lines indicate edges from 1 and stripped lines indicate edges from 2. We Add two coloured stripped circles on the 4 degree nodes to indicate the  $E_n^2$  edges.

#### Definition 1.2.10: RLeafD problem [7]

**Input:** A tuple  $(K, 0^k)$  such that  $k \in \mathbb{N}$  and  $K$  is a poly-time TM which describes a graph in  $G \in C_n$ . More specifically,  $K$  is a function  $V_{2^k} \rightarrow V_{2^k} \cup \{\perp\}$  such that

$$(a, b) \in E_n \iff K(a) = (\cdot, b) \wedge K(b) = (a, \cdot)$$

**Output:**  $v \in V_{2^k} \setminus \{0^n\}$  such that  $\exists a \in V_{2^k} : K(v) = (\cdot, a) \vee K(v) = (a, \cdot)$

**Embedding** ENDTOFLINE Chen et al. [7] demonstrated how every ENDTOFLINE can be reduced to a RLEAFD problem. Key insight, is the observation that every planar embedding of a graph of an ENDTOFLINE has the following property:

$$\forall v \in V_n : \deg(v) \in \{(0,0), (1,1), (1,0), (0,1), (2,2)\}$$

Using as reference the figure 1.2.3, we remove all the edges of nodes with degree of  $(2,2)$ . This in a way creates a different graph but the key insight is that the number of sources or sinks remain the same as we are essentially redirecting the lines as we can see from the figure 1.2.5. We will use these reductions in later sections to demonstrate reductions with excess problems as well.

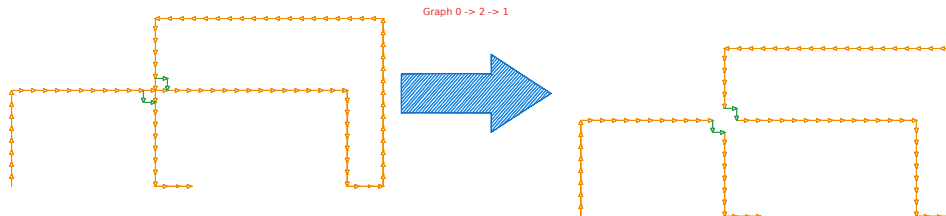


Figure 1.2.5: Depiction of the edge removal. Main idea is that this we keep the number of sources or sinks as well as invertible (we can trace the source or sink from the original instance), despite losing a lot of graph information.

**The PureCircuit problem** The *PureCircuit* problem is a discretised version of the  $\epsilon$ -GCircuit which was first introduced by Chen et al. [15, 8]. General idea of  $\epsilon$ -GCircuit is that we have a directed graph  $T = (V, G)$  where  $V$  denote value nodes and  $G$  denote gate nodes. Each value node

is assigned a value in  $[0,1]$  and the gates denote can denote the standard set of arithmetic operations  $\{+, -, *\}$  with other nodes or some constant, or boolean operators  $\{\wedge, \vee, \neg, <, >, =\}$  with other nodes or some constant. The goal of the problem is to assign every problem with a value such that the assingment is correct up to a factor of  $\pm\epsilon$ . This problem was crucial to demonstrate *PPAD*-hardness for constant  $\epsilon > 0$  by Rubinstein [15, 32].

*PureCircuit* is the problem that we get when taking the limit  $\epsilon \rightarrow \infty$  of the  $\epsilon$ -*GCircuit* problem. This results in a circuit which use the *Kleene valued logic* which extends the traditional binary logic [26]. We will explore the above algebra in greater depth in section ... This problem was created by Deligkas et al. [15] to demonstrate the hardness of approximating *PPAD* problems and was shown to be *PPAD*-COMPLETE.

**Definition 1.2.11: PURECIRCUIT Problem Definition [15]**

An instance of *PureCircuit* is given by vertex set  $V = [n]$  and gate set  $G$  such that  $\forall g \in G : g = (T, u, v, w)$  where  $u, v, w \in V$  and  $T \in \{\text{NOR}, \text{Purify}\}$ . Each gate is interpreted as:

1. *NOR*: Takes as input  $u, v$  and outputs  $w$
2. *Purify*: Takes as input  $u$  and outputs  $v, w$

And each vertex is ensured to have  $\text{in-deg}(v) \leq 1$ . A solution to input instance  $(V, G)$  is denoted as an assignment  $\mathbf{x} : V \rightarrow \{0, \perp, 1\}$  such that for all gates  $g = (T, u, v, w)$  we have:

1. *NOR*:

$$\begin{aligned} \mathbf{x}[u] = \mathbf{x}[v] = 0 &\implies \mathbf{x}[w] = 1 \\ (\mathbf{x}[u] = 1 \vee \mathbf{x}[v] = 1) &\implies \mathbf{x}[w] = 0 \\ \text{otherwise} &\implies \perp \end{aligned}$$

2. *Purify*:

$$\begin{aligned} \forall b \in \mathbb{B} : \mathbf{x}[u] = b &\implies \mathbf{x}[v] = b \wedge \mathbf{x}[w] = b \\ \mathbf{x}[u] = \perp &\implies (\mathbf{x}[v], \mathbf{x}[w]) \in \{(0, \perp), (\perp, 1), ((0, 1))\} \end{aligned}$$

In addition to the gates above, we will make use of the standard set of gates  $\{\vee, \wedge, \neg\}$ , whose behaviour is depicted in the tables 1.2.1. Moreover, we will make use of the *Copy* gate which we can define as  $\text{Copy}(x) = \neg(\neg x)$ . These gates are well defined based on the *NOR* gate as shown by Deligkas et al. [15] and do not directly affect the complexity of the problem. Furthermore, with respect to the *Purify* gate, the original problem states that one of two outputs must contain a pure value. We restrict the solution set to the one written in the definition as Deligkas et al. [15] showed that the only *Purify* solutions essential for *PPAD*-completeness are  $\{(0, \perp), (\perp, 1), (0, 1)\}$ , and the addition of more solutions does not affect its complexity.

| not     |         | and     | 0 | $\perp$ | 1       | or      | 0       | $\perp$ | 1 |
|---------|---------|---------|---|---------|---------|---------|---------|---------|---|
| 0       | 1       | 0       | 0 | 0       | 0       | 0       | 0       | $\perp$ | 1 |
| $\perp$ | $\perp$ | $\perp$ | 0 | $\perp$ | $\perp$ | $\perp$ | $\perp$ | $\perp$ | 1 |
| 1       | 0       | 1       | 0 | $\perp$ | 1       | 1       | 1       | 1       | 1 |

(a) not gate      (b) and gate      (c) or gate

Table 1.2.1: Three-valued logic [26]

## Topological problems

This section refers to types of problems which take advantage of some combinatorial property of their topology in order to guarantee a solution.

**Sperner Problems** Below we will refer to the notion of Sperner problems, which are types of problems that are based around *Sperner's Lemma*. The idea is as follows, given a triangle that has subdivided into smaller subtriangles, we first define the index of each point using the following set:

$$\forall n \in \mathbb{N} : \mathcal{T}_n \triangleq \{(x, y, z) \in [n] \mid x + y + z = n\}$$

We say  $\lambda : \mathcal{T}_n \rightarrow [3]$  is a valid colouring if and only if it satisfies the following conditions:

$$\forall (x_1, x_2, x_3) \in \mathcal{T}_n : \lambda(x) \in \begin{cases} \{2, 3\} & \text{if } x_1 = 0 \\ \{1, 3\} & \text{if } x_2 = 0 \\ \{1, 2\} & \text{if } x_3 = 0 \end{cases}$$

Given a valid colouring  $\lambda$ , we can always find a small triangle  $\tau$  where all 3 colours are assigned to one of its vertices. A visual example can be seen in figure 1.2.6.

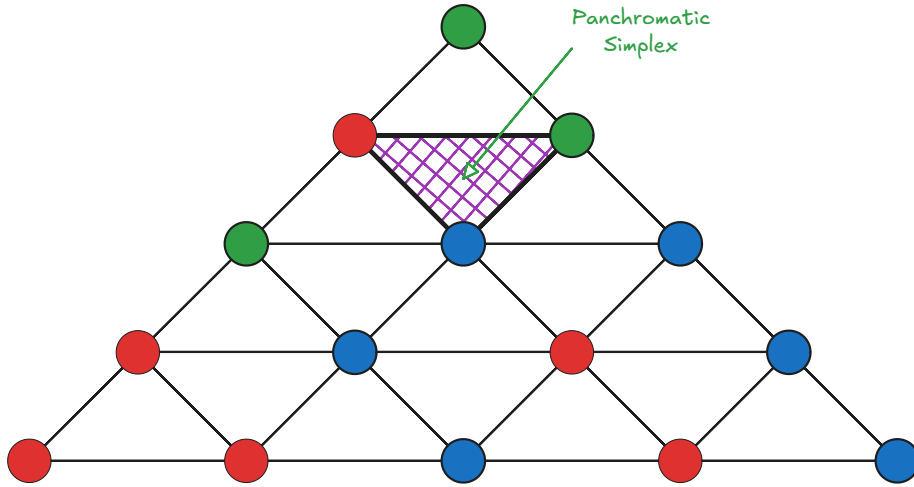


Figure 1.2.6: A colouring example of a subdivided simplex. As we can observe, no matter how we modify colourings, there will always be a small simplex that contains all colours

This type colouring will be referred to as the **linear** colouring where for dimension  $d$ , assign  $d + 1$  distinct colours to each point [11, 7], and it was shown that one can define a **PPAD**-complete variant of this lemma property. There several variants of the problem, where we can either work in a triangle with an exponential amount of subdivisions or polynomial subdivision but on higher dimensional simplexes. Both of these problems are **PPAD**-complete.

We introduce an alternative variant of the Sperner problems, introduced by Daskalakis et al. [13], known as *StrongSperner* or *BiSperner*. Instead of assigning  $d + 1$  colours to find a panchromatic simplex, each vertex is assigned  $d$  colours rather than one. We refer to the figure for a visual interpretation of the colouring 1.2.7. Essentially for each point in some hypercube  $[N]^d$ , each vertex is assigned  $d$  colours, where for colour  $i$  have choices either  $\{i^+, i^-\}$ . The goal is to find a cube where in every dimension we have both colours. This has been used in many problems throughout literature as seen in the papers [8, 15, 13] to demonstrated *PPAD*-hardness of many problems.

We will refer to this type of colouring as **bipolar** colouring, which has been used in grid-like topologies of the Sperner property We note that these terms are not standard in literature; we adopt this naming convention for of clarity.

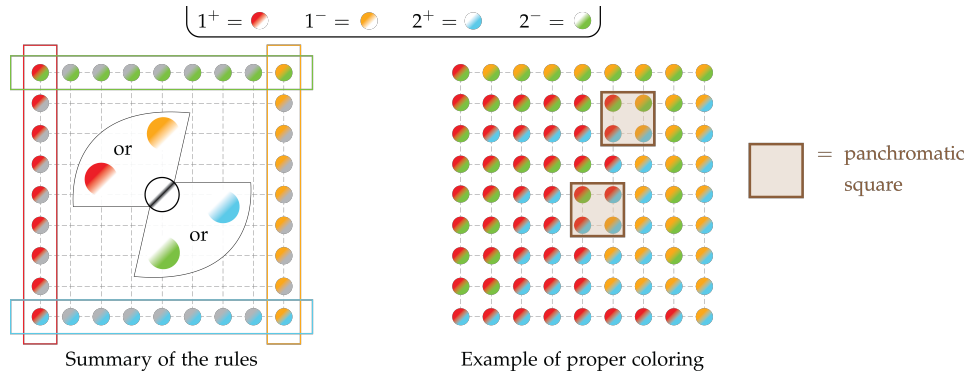


Figure 1.2.7: Visual interpretation of the *StrongSperner* problem. Figure from paper [12]

### Definition 1.2.12: Bipolar colouring

Given a set  $S^d$ , where  $S$  is some arbitrary set, we refer to bipolar colouring  $C$  as:

$$\forall v \in S^d, j \in [d] : [C(v)]_j \in \{-1, 1\}$$

We say that a set of points  $A \subseteq S^d$  **cover all the labels** if:

$$\forall i \in [d], \ell \in \{-1, +1\}, \exists x \in A : [\lambda(x)]_i = \ell$$

Deligkas et al. [15, 14] showed that  $\forall A \subseteq S^d : |A| \geq d + 1$  and satisfies the colouring,  $\exists D \subseteq A : |D| \leq d$  where  $D$  also satisfies the colouring condition.

### Definition 1.2.13: STRONGSPERNER problem

**1.2.13 Input:** A boolean circuit that computes a bipolar labelling  $\lambda : [M]^N \rightarrow \{-1, 1\}^N$  1.2.1 satisfying the following boundary conditions for every  $i \in [N]$ :

- if  $x_i = 1 \implies [\lambda(x)]_i = +1$
- if  $x_i = M \implies [\lambda(x)]_i = -1$

**Output:** A set of points  $\{x^{(i)}\}_{i \in [N]} \subseteq [M]^N$ , such that:

- *Closeness condition:*  $\forall i, j \in [N] : \|x^{(i)} - x^{(j)}\|_\infty \leq 1$
- *Covers all labels* as defined in 1.2.1

The above is a generalised variant of the traditional Sperner problem to a grid of dimensions  $N$  with width of  $M$ . Throughout literature the same variants of the problem or specifications have been defined using the names Sperner or discrete Brouwer [8, 7, 11, 15]. This is very similar to a different type of problem known as *StrongTucker*, which can be thought of as the *PPA* analog of *StrongSperner*. The two problems satisfy different boundary conditions but due to their similar labelling nature, we will borrow some techniques used in later sections. For clarity, the subsequent analysis adopts *StrongSperner*.



**Definition 1.2.14: ND-STRONGSPERNER problem**

**Input:** A tuple  $(\lambda, 0^k)$  of a STRONGSPERNER instance but for only  $n$  dimensions, such that  $\lambda : (\mathbb{B}^k)^n \rightarrow \{-1, +1\}^n$ .

**Output:** A point  $\alpha = (a_1, \dots, a_n) \in (\mathbb{B}^k \setminus \{1^k\})^n$  such that

$$\{\alpha + x \mid x \in \mathbb{B}^n\} \text{ covers all the labels } \textcolor{brown}{1.2.1}$$

We assume dimensionality  $n \geq 2$ .

The authors refer to both colouring schemes interchangeably when discussing their reductions, so we can assume that all reductions (not necessarily counting reductions) work similarly for either colouring [8, 15, 11, 7].

Other than topological colouring, we will also give a small introduction, to a tiling problem known as mutilated chessboards. Tiling spaces has been used throughout literature [19, 5]. We will use as reference the PPAD-complete 1-MC problem. We refer to the problem introduced by Blank [2] as follows: assume we have a  $2n \times 2n$  chessboard. If we are provided with  $1 \times 2$  pieces, we can trivially tile the board by filling each row with  $n$  horizontal pieces. The idea is when we remove the bottom left corner, the board becomes impossible to tile. The argument can be thought of as such: each tile piece covers a bright square and an odd square, which implies that it has to be the case that one of the squares has to remain untiled. An example can be seen in the figure 1.2.8. More formally we use the following definition 1.2.1, and Hollender et al. [19] demonstrated that this problem

**Definition 1.2.15: 1-MC [19]**

**Input:** Given a circuit  $C : 2^{2n} \rightarrow 2^{2n}$ , such that  $C(0,0) = (0,0)$ , such that tiling between two squares is indicated such that:

$$\forall x \in \{0,1\}^{2n} : C(C(x)) = x \wedge x - C(x) \in \{(0,1), (1,0), (0,-1), (-1,0)\}$$

**Output:** We want to find a point  $x \in \{0,1\}^{2n} \setminus \{(0,0)\}$  such that we have no tiling.

Lastly, we will introduce TARSKI 1.2.17 which uses the Knaster-Tarski fixed point theorem 1.2.1 and belongs in  $\text{PLS} \cap \text{PPAD}$ , where PLS is a class of problems based on the idea of local search [25].

**Definition 1.2.16: Monotone functions**

Given two partially ordered sets  $(L_1, \leq_{L_1})$  and  $(L_2, \leq_{L_2})$ , a function  $f : L_1 \rightarrow L_2$  is **monotone** if and only if:

$$\forall x, y \in L_1 : x \leq_{L_1} y \implies f(x) \leq_{L_2} f(y)$$

**Theorem 1.2.1: Knaster Tarski Fixed point theorem[3, 17]**

Given a *monotone* function  $f : L \rightarrow L$  1.2.16 of a complete lattice  $(L, \wedge, \vee)$ ,  $\exists c \in L : f(c) = c$ .

**Definition 1.2.17: TARSKI problem definition [17]**

Given a *monotone* function  $f : L \rightarrow L$  1.2.16 of a lattice  $(L, \wedge, \vee)$  we define solutions to the problem as:

1. Find  $x \in L : f(x) = x$
2. Find  $x, y \in L$  such that  $x \leq y$  and  $f(x) \not\leq f(y)$

We assume that  $f$  is represented by boolean circuits.



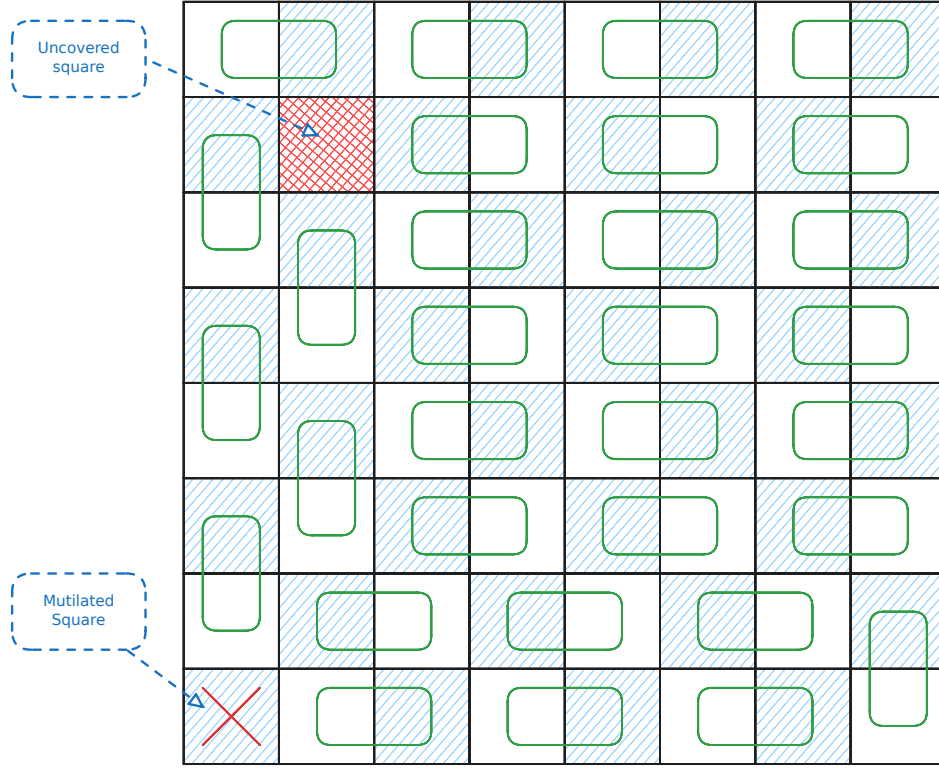


Figure 1.2.8: Example on an  $8 \times 8$  chessboard with a missing square. As we can observe, there will always be one square we cannot tile

**Counting complexity of PPAD** Ikenmeyer et al. [23] demonstrated that several PPAD-COMPLETE problems behave differently under  $\#PPAD - 1$ . More specifically  $\#ENDOFLINE - 1 \subseteq \#P$  but  $\exists A : \#SOURCEOREXCESS^A - 1 \not\subseteq \#P^A$ . We estimate the counting difficulty of the aforementioned problems based on their position in the hierarchy as seen in the figure ??.

## 1.2.2 Counting Complexity and Combinatorial Interpretations

Decision problems are problems that ask whether an input problem has a solution. Search problems as we observed earlier ask what is a valid solution given the current input instance like what is a valid colouring of a graph, satisfying assignment to a 3-SAT problem etc. Counting complexity, or counting problems take it a step further and usually ask how many valid solutions does a problem have. Valiant formalised this idea with the help of  $\#P$  complexity class 1.2.2, in which he demonstrated that even if a problem is polynomially solvable in its decision/search variant, can be much harder. A much simpler example can be found in the book by Barak et al. [1] where they provide a reduction from HAM-CYCLE to  $\#CYCLE$ .

### Definition 1.2.18: $\#P$ Complexity Class [33]

A function  $f : \mathbb{B}^* \rightarrow \mathbb{N} \in \#P$ , if there exists a poly-function  $p : \mathbb{N} \rightarrow \mathbb{N}$  and a poly-time TM  $M$  such that:

$$f(w) = \left| \left\{ v \in \mathbb{B}^{p(|w|)} \mid M(w, v) = 1 \right\} \right|$$

Alternative but equivalent definition is as follows: A function  $f \in \#P$ , if there exists a non-deterministic poly-time TM  $N$  such that:

$$f(w) = \#acc_N(w)$$

Where  $\#acc_N(w)$  denotes the number of accepting paths of  $N$  on  $w$ .

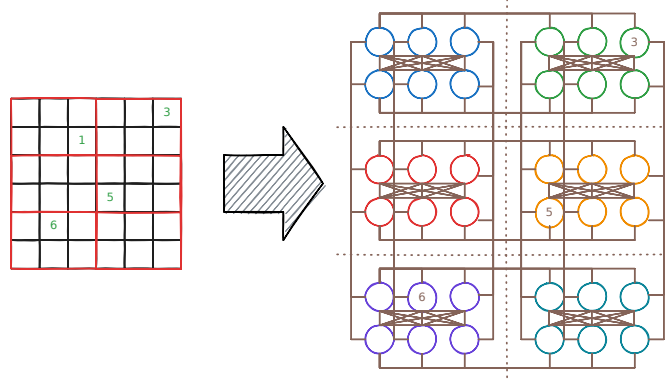


Figure 1.2.9: Example of reducing a 6x6 sudoku puzzle, into a graph colouring problem

In hindsight, we can observe that functions in  $\#P$  express functions of the form  $\forall w \in \{0,1\}^* : f(w) = |A_w|$ , where  $A_w$  is a set of objects of size polynomial to the input. Given the above definition, Pak et al. [30, 24] used  $\#P$  to decide whether a class of numbers or objects have or do not have a combinatorial interpretation. Moreover, they argue that the usage of  $\#P$  has several benefits:

1. By polynomially bounding objects, we avoid cases such as:  $f(w) = |\{1, \dots, f(w)\}|$ . In other words, we create a set of combinatorial objects.
2. We can work with  $f(\cdot)$ , even if its direct computation is hard.
3.  $\#P$  is a formal system that is highly flexible.

Lastly, we introduce the idea of correlating two counting problems with the help of *parsimonious reductions*. Simply, we say that a problem  $A$  can be parsimoniously reduced to a problem  $B$ , if for inputs of  $A$ , one can create instances of  $B$  such that the set of solutions between the two problems have the same cardinality. More formally we refer to the definition 1.2.19. An example of how such reduction may look like, can be seen in figure 1.2.9.

#### Definition 1.2.19: Parsimonious reductions

Let  $R, R'$  be search problems, and let  $f$  be a reduction of  $S_R = \{x \mid R(x) \neq \emptyset\}$  to  $S_{R'} = \{x \mid R'(x) \neq \emptyset\}$ . We say  $f$  is **parsimonious** if:

$$\forall x \in S_R : |R(x)| = |R'(f(x))|$$

Where for relation  $R$  we say  $R(x) \triangleq \{y \in \{0,1\}^* \mid xRy\}$

We introduce a more relaxed variant of parsimonious reductions in 1.2.2 for one-to-many reductions, such that the two solutions sets are bounded by some polynomial with respect to the size of the input. We observed that for several problems, it can be difficult to obs

#### Definition 1.2.20: Poly-Function Bounded Parsimonious Reductions

Given two counting problems  $A, B : \mathbb{B}^* \rightarrow \mathbb{N}$  and a poly-function  $f : \mathbb{N} \rightarrow \mathbb{N}$ , we say that:

$$A \subseteq^f B \iff \forall x \in \mathbb{B}^n : A(x) \leq B(x) \leq f(|x|) \cdot A(x)$$

If  $\forall x : f(x) = c$  for  $c \in \mathbb{N}$ , we use the abbreviation:

$$A \subseteq^c B$$

### 1.2.3 Kleene Logic and Hazard-Free Circuits

Kleene logic uses the boolean system with an additional *unstable* value [26]. The study of Kleene logic assisted in the development of robust physical systems [18] and aiding in the development of lower bounds for monotone circuits [16, 20, 21, 4]. The current section offers a brief summary to relevant notions and concepts in Kleene logic.

#### Definition 1.2.21: Kleene value ordering [29]

The instability ordering for  $\leq^u$  is defined as:  $\perp \leq^u 0, 1$ . The  $n$ -dimensional extension of the ordering is:

$$\forall x, y \in \mathbb{T}^n : x \leq^u y \implies \forall j \in [n] : (x_j \in \mathbb{B} \implies x_j = y_j)$$

#### Definition 1.2.22: Kleene Resolutions [29, 20]

Given  $x \in \mathbb{T}^n$ , we define the *resolutions* of  $x$ , using the following notation:

$$\text{RES}(x) \triangleq \{y \in \mathbb{B}^n \mid x \leq^u y\}$$

Unless otherwise specified, we assume that all Kleene functions are *natural* 1.2.23, since they can be represented by circuits 1.2.1. Detecting hazards is a concept that is analysed heavily when talking about Kleene logic. The idea is ensuring robustness in our circuits, meaning if all resolutions of  $x \in \mathbb{T}^n$  give the same value, then we should expect the circuit to behave the same way 1.2.24. An example of hazard values can be seen in the figure ??.

#### Definition 1.2.23: Natural functions [20]

A function  $F : \mathbb{T}^n \rightarrow \mathbb{T}^m$  for some  $n, m \in \mathbb{N}$  is *natural* if and only if it satisfies the following properties:

1. *Preserves stable values*:  $\forall x \in \mathbb{B}^n : F(x) \in \mathbb{B}^m$
2. *Preserves monotonicity* 1.2.16 1.2.22

#### Proposition 1.2.1: Natural functions and Circuits [29, 20]

A function  $F : \mathbb{T}^n \rightarrow \mathbb{T}^m$  can be computed by a circuit iff  $F$  is *natural* 1.2.23

#### Definition 1.2.24: Hazard [20, 16]

A circuit  $C$ , on  $n$  inputs has **hazard** at  $x \in \mathbb{T}^n \iff C(x) = \perp$  and  $\exists b \in \mathbb{B}, \forall r \in \text{RES}(x) : C(r) = b$ . If such value does not exist then we say that  $C$  is hazard-free.

#### Definition 1.2.25: K-bit Hazard [20]

For  $k \in \mathbb{N}$  a circuit  $C : \mathbb{T}^n \rightarrow \mathbb{T}$  has a *k-bit hazard* at  $x \in \mathbb{T}^n \iff C$  has a hazard at  $x$  and  $\perp$  appears at most  $k$  times.

We also wish to introduce the following corollary by Ikenmeyer et al. [20] which allows us to construct  $k$ -bit hazard free circuits

**Corollary 1.2.1: K-bit Hazard Construction [20]**

Given circuit  $C : \mathbb{T}^n \rightarrow \mathbb{T}$ , one can construct a  $k$ -bit hazard free circuit of  $C$  denoted as  $C'$  such that:

$$\begin{aligned} \text{size}(C') &= \left(\frac{ne}{k}\right)^k (\text{size}(C) + 6) + O(n^{2.71k}) \\ \text{depth}(C') &= \text{depth}(C) + 8k + O(k \log n) \end{aligned}$$

Where  $\text{size}(P)$  denotes the number of gates of  $P$  and  $\text{depth}(P)$  denotes the longest path from the input bit to the output bit

**Kleene logic and Complexity Theory**

Lastly we want to introduce the notion of *Alternating Turing Machines*, which are meant to represent a generalisation of non-deterministic Turing machines. These have been introduced by Chandra et al. and the idea is that we introduce universal and existential quantifiers to alternate the course of computation. The informal presentation can be thought as follows: In a non-deterministic TM, given a single state  $q$  or configuration  $c$ , we execute  $\beta_1, \dots, \beta_k$  separate processes. We say that  $q$  will accept if at least one of the processes  $\{\beta_i\}_{i \in [k]}$  accepts. Alternating Turing machines introduce additional conditions where one accepts all  $\{\beta_i\}_{i \in [k]}$  to accept or in case of a negation state, if  $\beta_1$  accepts, then  $q$  rejects or vice versa. More formally, we offer the definition in 1.2.3.

**Definition 1.2.26: Alteranting Turing Machines(ATM) [27, 6]**

An alternating TM is a seven tuple

$$M = (k, Q, \Sigma, \Gamma, \delta, q_0, g)$$

Where each term denotes:

1.  $K$ : The number of work tapes
2.  $Q$ : Finite set of states
3.  $\Sigma$ : *Finite* input alphabet. Moreover,  $\star \notin \Sigma$  denotes the end of the input
4.  $\Gamma$ : *Finite* work tape alphabet. Moreover  $\# \in \Gamma$  to denote blank symbol.
5.  $\delta$  transition relation which is defined as:

$$\delta \subseteq \left( Q \times \Gamma^k \times (\Sigma \cup \{\star\}) \right) \times \left( Q \times (\Gamma \setminus \{\#\})^k \times \{L, R\}^{k+1} \right)$$

Important to observe that for a single state, input symbol and work tape symbols, we can take multiple types of transitions.

6.  $q_0 \in Q$ : Is the initial state
7.  $g : Q \rightarrow \{\vee, \wedge, \neg, \mathbf{1}, \mathbf{0}\}$ : Where  $g$  denotes the type of state and  $\mathbf{1}, \mathbf{0}$  denote whether it is accepting or not.

The machine essentially contains a read-only input tape with endmarkers and  $k$  work tapes which are initially blank. A *step* of  $m$  consists of reading a symbol from the input tape, writing a symbol to each work tape, moving the writing head in each head left or right while transitioning into a new state.

**Definition 1.2.27: ATM configurations [27, 6]**

A configuration  $\mathcal{C}$  of an ATM  $M$  is a element of the set  $\mathcal{C}_M$

$$\mathcal{C}_M \triangleq Q \times \Sigma^* \times (\Gamma \setminus \{\#\})^k \times \mathbb{N}^{k+1}$$

Which represent, the current state, the input, the current contents of each work tape, and the  $k + 1$  head positions.

We say that  $\mathcal{C}$  that *configuration* is universal (existential, negating, accepting or rejecting) if  $q$  is also universal (existential, negating, accepting or rejecting). The initial configuration of  $M$  on input  $x$  is

$$\sigma_M(x) = (q_0, x, \lambda^k, 0^{k+1})$$

Where  $\lambda$  is a null string.

**Definition 1.2.28: Successor of configurations [6]**

Given  $M$ , we write  $\mathcal{C} \vdash \mathcal{C}'$  to say that  $\mathcal{C}'$  is the successor of  $\mathcal{C}$ , if we can go from one configuration to another with a single  $\delta$  rule. We require for  $\delta$  that, for accepting or rejecting configurations to have no successors, for universal or existential to have at least one, and for negating configurations to have only one. We can define the **computation** of a path on  $x$  as a sequence  $a_0 \vdash a_1 \vdash \dots \vdash a_n$  where  $a_0 = \sigma_M(x)$

**Denoting Accepting or Rejecting computations** As mentioned previously, we start from the original configuration. If a process is in an existential configuration  $\mathcal{C}$  and  $\beta_1, \dots, \beta_m$  are all possible successors, then we spawn a process for each configuration and run each one concurrently. If at least one process is accepting we report back to the parent and terminate the rest. Same line of reasoning can be done with the universal quantifiers. Ideally we would like a labelling function  $f$  which is defined as

$$f(\mathcal{C}) = \begin{cases} \bigwedge_{\mathcal{C} \vdash \beta} f(\beta) & \text{if } \mathcal{C} \text{ is universal} \\ \bigvee_{\mathcal{C} \vdash \beta} f(\beta) & \text{if } \mathcal{C} \text{ is existential} \\ f(\beta) & \text{where } \mathcal{C} \vdash \beta \text{ if } \mathcal{C} \text{ is negating} \\ \mathbf{1} & \text{if } \mathcal{C} \text{ is accepting} \\ \mathbf{0} & \text{if } \mathcal{C} \text{ is rejecting} \end{cases}$$

But some processes may never halt or require a much greater computation than necessary. We therefore use Kleene logic  $\mathbb{T}$  where  $\perp$  denotes configurations that we do not know if they are accepting or not. The quantifiers use the same logic as described in the tables 1.2.1. We now say that a labelling of configurations is a map  $\lambda : \mathcal{C}_M \rightarrow \{0, \perp, 1\}$ . We define an operator  $\tau$  to be an operator of mappings such that:

$$\tau(\ell)(\mathcal{C}) = \begin{cases} \bigwedge_{\mathcal{C} \vdash \beta} \ell(\beta) & \text{if } \mathcal{C} \text{ is universal} \\ \bigvee_{\mathcal{C} \vdash \beta} \ell(\beta) & \text{if } \mathcal{C} \text{ is existential} \\ \ell(\beta) & \text{where } \mathcal{C} \vdash \beta \text{ if } \mathcal{C} \text{ is negating} \\ \mathbf{1} & \text{if } \mathcal{C} \text{ is accepting} \\ \mathbf{0} & \text{if } \mathcal{C} \text{ is rejecting} \end{cases}$$

We can check that  $\tau$  is monotone with respect to  $\leq$  since if  $l \leq l'$ , then  $\tau(l) \leq \tau(l')$ . By 1.2.1 there

must exist maximum labelling such that

$$\ell^* = \sup_{m \in \mathbb{N}} \tau^m(\Omega)$$

where the supremum is with respect to  $\leq$ , where

$$\begin{aligned} \tau^0(\ell) &= \ell \\ \tau^{m+1}(\ell) &= \tau(\tau^m(\ell)) \end{aligned}$$

where  $\Omega$  is denoted as the minimal labelling function  $\lambda x. \perp$ . In a way that the above procedure describes is the "propagation" process from the leaf nodes up to the root of the tree where the leaf nodes are assigned a label of  $\perp$  or accepting. An example can be seen in the figure. Lastly given the above we can finally give a definition of halting states

**Definition 1.2.29: Halting states[6]**

We say that  $M$  halts on  $x$  iff  $\lambda^*(\sigma_M(x)) \in \{0, 1\}$  and it accepts or rejects respectively iff  $\lambda^*(\sigma_M(x)) = 1$  or  $\lambda^*(\sigma_M(x)) = 0$

We aim for this ATM to work for only recursively enumerable sets and therefore we aim to limit to the depth of the recursion. Therefore, we define operator  $\tau_C$  where  $C \subseteq C_M$  such that

$$\tau_C(\ell)(a) \triangleq \begin{cases} \tau(\ell)(a) & \text{if } a \in C \\ \perp & \text{otherwise} \end{cases}$$

Where we can say that  $C$  is the set of configuration that are reachable within some  $f(|x|)$  steps or require at most  $f(|x|)$ . An example of a computation tree can be observed in the figure 1.2.10.

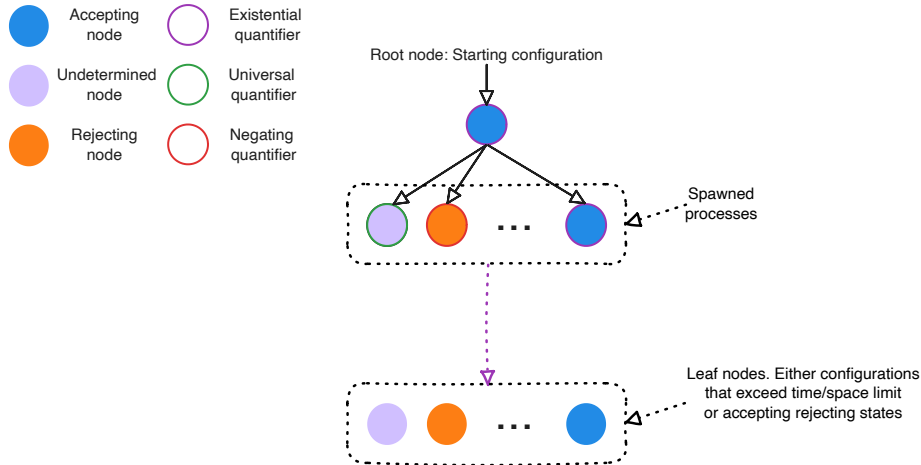


Figure 1.2.10: Computation tree of an ATM with labellings

We can observe that one can encode Kleene logic into the computation of Turing machines. We use this framework as one of the main motivations of the project.

### 1.3 Literature Review

Our work is primarily focused around the paper by Ikenmeyer et al. [23] and the parsimonious reductions discovered across various **PPAD** problems. We introduce the notation  $\#\mathbf{PPAD}(L)$ , where this indicates the class of problems which are parsimoniously reducible to  $L$ . In fact, we

extend this notation with  $f(\#PPAD(L))$  to indicate reductions of the form, given  $L'$ , we say that  $L' \in f(\#PPAD(L))$  if

$$\forall x \in S_{L'} : |R_{L'}(x)| = f(|R_L(g(x))|)$$

We demonstrate an example of the above notation using a finding from the paper by Ikenmeyer et al. [23]:  $PPAD(EndOfLine - 1)/2 = \#P$ . To show inclusion it is easy to see that one can accept all the non-zero sources, since each source is paired with a sink, and we are able to ignore the sink paired with the  $0^n$  source. To show the other direction, they demonstrate an example where they reduce  $CircuitSAT \leq_C (\#EndOfLine - 1)/2$  by creating a graph, where for each input  $x$ , we create a source and sink pair  $(0x, 1x)$ .

Ikenmeyer et al. [23] developed tools to demonstrate how can one show whether a problem is in  $\#P$  or not, or more specifically, how do demonstrated whether certain functions exhibit combinatorial interpretations. In their paper, they established that for some  $PPAD$  problems,  $\#PPAD(\cdot) - 1 \subseteq \#P$  but for some other this statement did not hold true under relativising reductions, such as the *SourceOrExcess*. The aforementioned statement oracle separation is known as an oracle separation. The details of how it works are beyond the scope of the current project and therefore we focus on the key takeaway that  $\#PPAD(SourceOrExcess(2, 1)) - 1$  is a harder problem than  $\#PPAD(EndOfLine) - 1$ .

But one can ask, how does  $\#PPAD(SourceOrExcess(k, 1)) - 1$  behave for some  $k \in \mathbb{N}_{\geq 3}$  and in general we will demonstrate that there are many other problems that have interesting counting properties. This leads us directly to the question of, can we somehow quantify each problem in  $PPAD$ , or more specifically, is there a problem that can “everything” else?

### 1.3.1 Project Question and Objectives

As mentioned in the previous section, we observed how different  $PPAD$  problems can behave differently. The main motivation comes from the observation of how *Kleene logic* can be used to simulate the computation paths of NTMs. We can parallelise this to the boolean problem of *SAT* which given the *Cook-Levin* theorem [10] one can convert every  $NP$  problem into a *SAT*(problem of satisfying assignments for boolean expression) problem by creating a boolean expression for each snapshot of the Turing Machine such that it returns an accepting state. Another property of the *Cook-Levin* is that every problem can be parsimoniously reduced to a *SAT* assignment, meaning:

$$\forall L \in NP : \#NP(L) \subseteq \#NP(SAT)$$

A reasonable conjecture could also be if one can exploit Kleene-circuit nature of *PureCircuit* to create a *Cook-Levin* type equivalent reduction for  $PPAD$  problems, or more formally:

$$\forall L \in PPAD : \#PPAD(L) \subseteq \#PPAD(PureCircuit)$$

If the above statement were indeed true, then it would give us insights as to how counting problems behave in  $PPAD$  as in the section .., we highlighted that even if two problems are  $PPAD$ -complete, it does not necessarily imply they are counting equivalent.

As established previously and as of the time of writing this paper, the *PureCircuit* was mainly established for inapproximability bounds of  $PPAD$  problems. Therefore, to tackle the aforementioned conjecture we established the following set of milestones.

1. Establish a chain of parsimonious reductions from the *EndOfLine* to the *PureCircuit* problem.
2. Establish a chain of parsimonious reductions from the *SourceOrExcess*(2, 1) to the *PureCircuit* problem. Ideally, we would like to extend this to the entire hierarchy of *SourceOrExcess*( $k$ , 1) problems for some  $k \in \mathbb{N}$  or ideally. We will introduce these problems more formally in the upcoming sections *PolySourceOrExcess*.

---

3. Establish a *Cook-Levin* type theorem for the all **PPAD** problems or more formally

$$\forall L \in \mathbf{PPAD} : \#\mathbf{PPAD}(L) \subseteq \#\mathbf{PPAD}(\text{PureCircuit})$$

To assist with the theory development, we set out to develop a visualisation tool that will be able to visualise instances of the problem as well as find solutions or enumerate through solutions for small instances. We will give a greater in-depth overview of the tool in the section .., its requirements and its current state.



## **Chapter 2**

### **Main findings**

## 2.1 Theory Analysis

In the current section, we explore on the approaches and findings we uncovered while tackling each of our theory objectives. We focus on relevant findings for each section, as well as a supplementary section for findings that do not relate with any of our objectives but are relevant with the concept of *PureCircuit* and **#PPAD**.

### 2.1.1 PureCircuit and EndOfLine

The core idea of our this current section comes from following set of observations: when working with *PureCircuit*, one cannot fully work with binary logic, meaning there is no trivial method for detecting  $\perp$  values and removing them. We therefore make use of topological problems with the idea that resolutions of our input corresponds to some set nearby points in a space. If all points follows abide some property then we can use that to detect solutions. We start with looking at the *StrongSperner* problem under the lens of *Kleene logic*. First we begin with the idea that solutions of *StrongSperner* problems revolve around the notion of panchromatic hypercubes. We can use the idea and make the following definitions

#### Definition 2.1.1: StrongSperner anchor points

Given a hypercube  $H \subset \mathbb{Z}^n$  of side lengths 1, we denote the  $\mathbf{Anc}(H)$  as

$$\mathbf{Anc}(H) = \min_{p \in H} \|p\|_1$$

Alternatively, we will refer to these hypercubes using the notation  $\mathbf{Cube}(\cdot)$  such that

$$\mathbf{Cube}(p) \triangleq \{(p + b) \mid b \in \{0, 1\}^n\}$$

Ideally, we would like to express a cube as a coordinate of Kleene numbers, implying the resolutions of the point will result in a hypercube, where each point is within  $\|\cdot\|_\infty \leq 1$  of each other. An example of that can be seen in figure 2.1.1. This leads to a natural interpretation of the *StrongSperner* problem in *Kleene logic*. We will start with two definitions, one for *nD-StrongSperner* and the other for *StrongSperner*. We remind that the main difference of the two problems is the former has fixed dimensions  $n$  of exponentially large cubes, whereas the other can have varying dimensions of polynomially large widths.

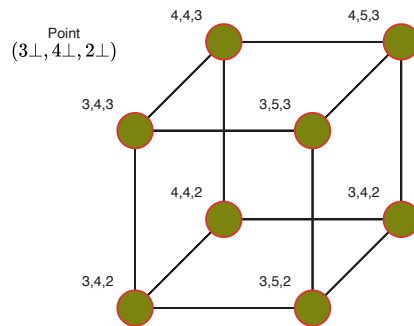


Figure 2.1.1

#### Definition 2.1.2: HF-nD-StrongSperner problem

**Input:** A circuit  $\lambda : (\{0, 1\}^k)^n \rightarrow \{-1, +1\}^n$ , that describes a *nD-StrongSperner* labelling, as explained in 1.2.13, such that it is hazard free on inputs  $(p_i \perp)_{i \in [n]}$  where  $p_i \in \{0, 1\}^{k-1}$  **Output:** A point  $p \in \times_{i \in [n]} (\{0, 1\}^{k-1} \times \{0, 1, \perp\})$  if and only if:  $\lambda(p) = \perp^n$ .

We argue that the above problem is *PPAD*-complete for all *HF- $nD$ -StrongSperner* problems. First we can observe that all *HF- $nD$ -SS* are reducible to their  *$nD$ -SS* counterparts since hazard-free transformations have no effect on binary inputs and therefore our circuits acts like any other boolean circuit. Chen et al. [8] demonstated that for any  $n \in \mathbb{N}$ ,  *$nD$ -StrongSperner* is **PPAD**-complete, and therefore *HF- $nD$ -StrongSperner* is in **PPAD**. To show the hardness, we claim that

$$\mathbf{PPAD}(nD\text{-}SS) \subseteq \mathbf{PPAD}(HF\text{-}nD\text{-}SS)$$

*Proof.* Given circuit  $\lambda : (\{0,1\}^k)^n \rightarrow \{-1,+1\}^n$  from  *$nD$ -SS*, assume that  $S$  is the set of anchors points of a panchromatic cube. We create  $\Lambda : (\{0,1\}^{k+1})^n \rightarrow \{-1,+1\}^n$  such that:

$$\Lambda((x_i)_{i \in [n]}) = \lambda \left( \left( \left\lfloor \frac{x_i + 1}{2} \right\rfloor \right)_{i \in [n]} \right) \quad (2.1.1)$$

We prove the following claim

**Claim 2.1.1: Transformation claim**

Given  $S_{\text{even}}$  the set of even solutions of the transformed space or more formally:

$$S_{\text{even}} \triangleq \left\{ (p_i 0)_{i \in [n]} \mid p \in (\mathbb{B}^{(k)})^n : (p_i 0)_{i \in [n]} \text{ is an anchor of a panchromatic cube} \right\}$$

We claim  $|S| = |S_{\text{even}}|$

*Proof.* We argue that we can bijectively map between  $S \leftrightarrow S_{\text{even}}$ . To show the left to right direction, assume  $p \in S$ . Using figure 2.1.2 as reference, we can observe that each point becomes a  $2^d$  hypercube. Essentially the hypercube that was anchored at  $(p_1, \dots, p_n)$  is mapped to the hypercube anchored at  $(2p_1, \dots, 2p_n)$ .

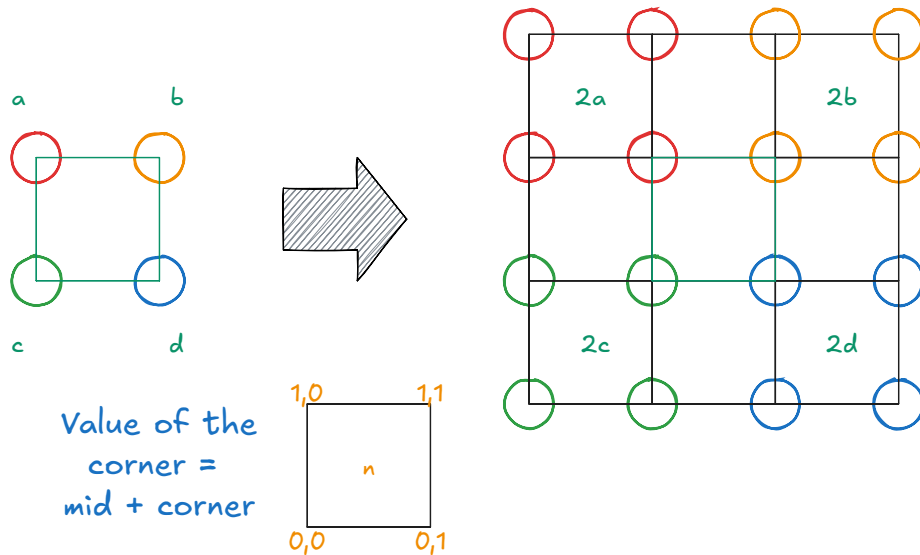


Figure 2.1.2: Dimension doubling. We demonstrate the example in two dimensions.

Moreover, we can make the following observation:

$$\begin{aligned}
\forall (2p_i)_{i \in [n]} \in (\{0, 1\}^{k+1})^n, b \in \{0, 1\}^n : \Lambda((2p_i + b)_{i \in [n]}) \\
&= \lambda \left( \left( \left\lfloor \frac{2p_i + b + 1}{2} \right\rfloor \right)_{i \in [n]} \right) \\
&= \lambda \left( \left( \left\lfloor p_i + \frac{b + 1}{2} \right\rfloor \right)_{i \in [n]} \right) \\
&= \lambda((p_i + b)_{i \in [n]})
\end{aligned}$$

Which implies that the cube anchored at  $p$  must also be panchromatic.  $\square$

To ensure that  $\Lambda$  is hazard free we can use the corollary 1.2.3, where we can create an  $n$ -bit hazard-free circuit. One must observe that even though, the size of the circuit increases exponentially with respect to the dimensions, but since all the input instances scale with respect to the width of the hypercube space, that would imply that we are able to keep the size of the circuit polynomial. We therefore claim that this reduction is polynomial and can parsimoniously find all panchromatic cubes.  $\square$

### Definition 2.1.3: Kleene Unary Numbers

Given a number  $p \in M$ , its unary representation can be depicted as a binary number  $\{0, 1\}^M$  such that:  $k \in M \rightarrow 1^k \in \bar{M}$ . Under Kleene algebra, we mainly refer to notations  $M_k^n$  where  $k, n, M \in \mathbb{N}_0$  such that:

$$M_k^n \triangleq \bigtimes_{i \in [n]} \{1^j \perp^k \mid \forall j \in \mathbb{N}_0 : j + k \leq M\}$$

For a cleaner notation, given  $1^j \perp^k \in M_k$  we denote it as  $j \perp^k$ .

For examples of the above notation we refer to the following table:

| Kleene Unary Sets | Member examples                          |
|-------------------|------------------------------------------|
| $3_1^2$           | $(11\perp, 1\perp 0) = (2\perp, 1\perp)$ |
| $5_0^3$           | $(11000, 11110, 11111) = (2, 4, 5)$      |
| $7_2^2$           | $11\perp\perp 000 = 2\perp^2$            |

Table 2.1.1: Kleene Unary Examples

### Definition 2.1.4: HF-STRONGSPERNER problem

**Input:** A hazard-free circuit  $\lambda : [M]^n \rightarrow \{-1, +1\}^n$ , where  $M \in n^{O(1)}$  that describes a *StrongSperner* labelling, as explained in 1.2.13, where the inputs are represented using Kleene Unary numbers 2.1.1.

**Output:** A point  $p \in M_{\leq 1}^n$  if and only if:  $\lambda(p) = \perp^n$ .

We will mainly focus on subproblems of the *HF-StrongSperner* where two cubes do not overlap with each other. We will use figure 2.1.3 for reference. Essentially if an edge or a face of a cube result in a solution then, it may cause two issues. One is that the More formally we define

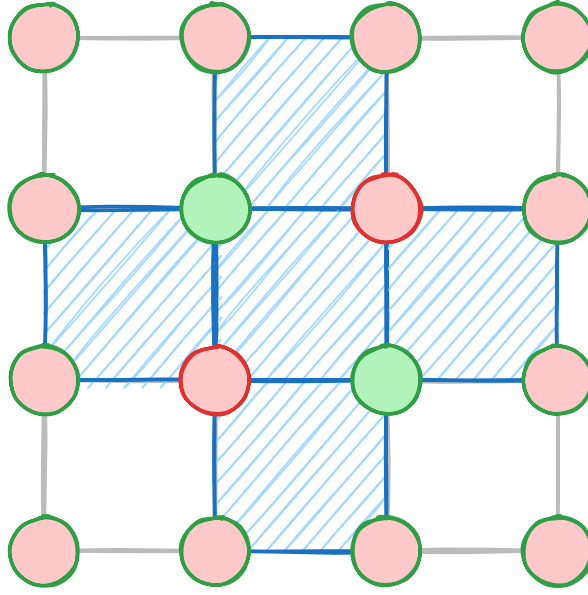


Figure 2.1.3: Overlapping Sperner Solutions

**Definition 2.1.5: HF-DISCRETESTRONGSPERNER problem**

An *HF-StrongSperner* instance  $\lambda$  such that  $\forall p \in [M_1]^n$ , if  $\lambda(p) = \perp^n$ , then

$$\forall y \in (\{0, 1\}^M)^n : x \leq y \implies y \text{ does satisfy the labelling condition } 1.2.1$$

Essentially, any subspace of the cube should not yield a panchromatic solution.

Lastly we introduce a specific subset of the above solutions where introduce antipodal panchromatic cubes. In the specific type of solution said we argue that two opposite corners of a cube contain opposite colourings. This notion of antipodal cubes is used in the majority of reductions across literature. More formally we say

**Definition 2.1.6: MinRes min and max resolution functions**

We use the **MinRes**, to denote the smallest resolution of a kleene string. More formally we can use the following equivalent definitions:

$$\forall x \in \{0, 1, \perp\}^*, j \in [|x|] : [\mathbf{MinRes}(x)]_j \triangleq \begin{cases} x_j & \text{if } x_j \in \{0, 1\} \\ 0 & \text{otherwise} \end{cases}$$

For the maximum resolution we will use the **MaxRes** function where we instead replace all  $\perp$  with 1 instead.

**Definition 2.1.7: HF-ANTIPODALSTRONGSPERNER problem**

An HF-DISCRETESTRONGSPERNER instance  $\lambda$  such that  $\forall p \in [M]^n$ , if  $\lambda(p) = \perp^n$ , and denote  $\text{MinRes}(x) = \hat{x}$  as the *anchor* of the cube, then:

$$\forall b \in \{0, 1\}^n, j \in [n] : [\lambda(\hat{x} + b)]_j = -[\lambda(\hat{x} + 1^n \oplus b)]_j$$

We emphasize that the above class considers the subset of the *Hf-DiscreteStrongSperner* as the antipodality condition cannot ensure discreteness.

**Claim 2.1.2: Antipodal discrete squares**

Given a panchromatic solution of a HF-ANTIPODALSTRONGSPERNER  $p$ , it must be the case then that:

$$\forall j \in \{-1, 1\}^n, \exists! x \in \text{Res}(p) : \lambda(x) = j$$

*Proof.* We proof by induction on  $n$ . Lets assume base case  $n = 2$ . We can observe that the only satisfying cube that is both antipodal and discrete, is some rotation of the following matrix

$$\begin{pmatrix} \mp & + \\ - & \pm \end{pmatrix}$$

Where we denote  $+$  =  $(+, +)$ ,  $-$  =  $(-, -)$ ,  $\pm$  =  $(+, -)$  and  $\mp$  =  $(-, +)$ , we can observe indeed that the panchromatic square contains all the colourings. For the induction step, we can observe that if we want our colouring function to be both discrete and antipodal, then the following must suffice: given  $k, k'$  the two resolutions of dimension  $k + 1$ , we know that  $[\lambda(p_{-(n+1)}(k))] = (b, \perp^n)$  for some  $b \in \{0, 1\}^n$ . But that implies  $[\lambda(p_{-(n+1)}(k'))] = (\bar{b}, \perp^n)$ , which indicates a discrete antipodal panchromatic solution. Moreover if  $p_{-(n+1)}$  cover  $2^n$  labels, then the whole cube covers all  $2^{n+1}$  labels, which by pigeonhole principle means that we must cover all possible combinations of colourings.  $\square$

All the problems above are clearly in **PPAD** as all of them reduce to the *StrongSperner* problem, since we assume by construction that they *natural* and therefore behave as normal binary circuits. For the purposes of our current research, we demonstrate a parsimonious reduction from HF-DISCRETESTRONGSPERNER to PURECIRCUIT.

**Theorem 2.1.1**

$$\#\text{PPAD}(\text{HF-DISCRETESTRONGSPERNER}) \subseteq \#\text{PPAD}(\text{PURECIRCUIT})$$

*Proof.* Given an input instance  $\Lambda$  of a HF-DISCRETESTRONGSPERNER, we create a PURECIRCUIT *Input nodes*, *Output nodes*, *Bit generator gadget* and *Circuit application gadget*. To visualise our construction we will refer to the figure in 2.1.4. Each of these parts describe the following:

1. **Input nodes:** Vector of  $n$  value nodes. Will be represented as  $(x_i)_{i \in [n]} \in \mathbb{T}^n$ .
2. **Bit generator gadget:** Given a value node  $x_i$ , apply the *Bit generator* gadget such that we create Kleene unary number as defined in 2.1.1, where  $x^i \in M_n^{\leq 1}$ . We will refer to the bit generator circuit as  $C_i$ .
3. **Circuit application gadget:** Since the above numbers form coordinates in a  $[M]_0^n$  space, we apply the  $\Lambda$  function to extract a set of output nodes  $(u^i)_{i \in [n]} \in \mathbb{T}^n$ .
4. **Output nodes:** We use the Copy gate to copy  $u^i \rightarrow x_i$ , for all  $i \in [n]$ .

The 1st and last part of our construction is self-explanatory, we will analyse the other two sections in greater depth and prove some claims.

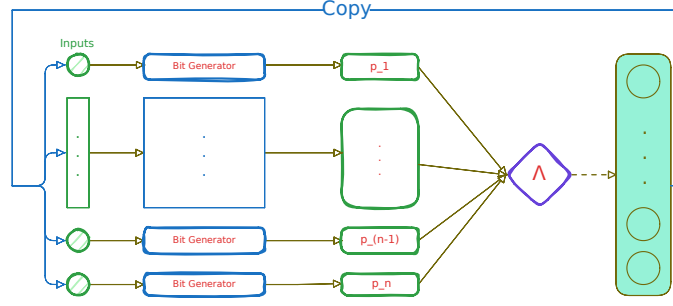


Figure 2.1.4: Pure Circuit construction visualisation

**Bit generator gadget** In order to create Kleene unary numbers we make use of a binary tree of Purify gates as seen in the figure 2.1.5. We now claim that the current construction has certain desirable properties.

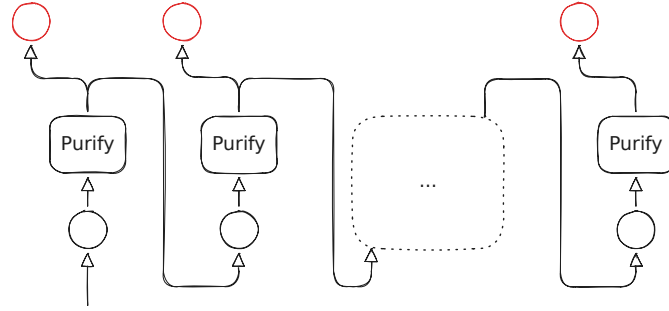


Figure 2.1.5: Purification gadget. The red nodes denote the outputs of the purification gadget.

### Claim 2.1.3: Purification Claim

Given the description of the bit generator gadget, we make the following two claims:

1.  $b \in \{0, 1\}^n : C_i(b) = b^n$
2.  $C_i(\perp) \in M_n^{\leq 1}$

*Proof.* From the definition of the Purify gate we know that  $\forall b \in \{0, 1\} : \text{Purify}(b) = (b, b)$ , and therefore any subtree that takes a pure input, will copy its value to all its leaves, which follows our first claim directly. To the second part of our lemma, we can observe that the only valid outputs are  $(0, \perp), (\perp, 1), (0, 1)$ . Based on our previous observation, if a purify gate outputs a  $(0, \perp)$  value, it will propagate  $\perp$  to the rest of the tree and return 0 in the current level, if the value is  $(\perp, 0, 1)$ , the tree will return  $\perp/0$  at the specific level, and then propagate 1 to the rest of the leaves. This shows that all possible strings generated are of the form  $1^k \perp^0 1 0^*$ .  $\square$

**Correctness and parsimonious argument** One can observe that the current construction is a valid construction, as we satisfy the arities of all gates and ensured that every node has an in-degree of 1. We now make the following lemma

### Lemma 2.1.1: Correctness Lemma

A correct assignment corresponds to panchromatic cube in the original problem instance.

It suffices to show that every vector  $x^i \in M_{\perp}^1$  and  $\forall i \in [n] : x[u^i] = \perp$ . Assume that we have a correct assignment, and WLOG  $x[u^i] = 0$  for some  $i \in [n]$ . By our copy that, we know that  $x[u^i] = x[x_i]$ . By our purification lemma .. we know that  $x[x^i] = 0^i$ . Since our circuit is hazard-free and applies the boundary conditions of the sperner problem, we know that  $[\Lambda(x_{-i}(0))]_i = 1 \neq x[u^i]$ .

We can make a similar argument for  $x[u^i] = 1$ . Therefore, a satisfying assignment corresponds to some panchromatic solution. Given that our instance is in **HF-DISCRETESTRONGSPERNER**, it has to be the case that all  $x_i \in M_n^1$ , otherwise that would imply that there exists a  $(n-1)$ -subspace that also covers all the labels. This also shows that there exists a 1-to-1 mapping between a panchromatic cube and a satisfying assignment.  $\square$

Although the above reduction also works for the general **HF-StrongSperner**, the reduction is not parsimonious since the *PureCircuit* will accept solutions for all  $(n-1)$  subspaces that contain a covering set.

Unfortunately, showing hardness is a much harder task. In the paper [8] they showed that given  $f(n) \in O(n)$ , they demonstrated how one can apply the *Snake Embedding* technique where one can fold the space in higher and higher dimensions with the goal of going from a  $2^n \times 2^n$  space to  $f(n)^m$  where  $m, f(n) \in O(n)$ . The original reduction, expresses the ability to go from a  $2^n \times 2^n$  space, to a  $[2^f(n)]^d$  where  $d = \lceil \frac{n}{f(n)} \rceil$  space, where  $f(n)$  is some well-defined polynomial function that grows linearly or less. We know that *EndOfLine* can parsimoniously reduce to the *2D-StrongSperner* problem, and as we will demonstrate in the later sections, using their snake embedding technique, the number of solutions will be preserved per folding. The problem is that we cannot guarantee, the hazard-freeness property of every fold. If we try to use the corollary 10 by [20], we observe that the circuit size will increase exponentially. This implies the best we can hope for when using their method is:

$$M = n^k, f(n) = \log_2(n)^k \implies d = \frac{n}{\log(n)^k}$$

In the upcoming sections, we propose a chain of parsimonious reductions from the *EndOfLine* to *AntipodalStrongSperner*. We emphasize that this method will still not be hazard-free, but we hope that the antipodal properties of resulting problem may enable for a full reduction in the future. We will start with the *Bipolar colouring* of *RLeafD*, where we extend the colouring of Chen et al. [7].

**Bipolar colouring of *RLeafD*** We construct the same reduction as in the *RLeafD* but with slight differences: We will use points  $\mathbf{u}, \mathbf{v}, \mathbf{w} \in V_{2^k}$  and  $\mathbf{p}, \mathbf{q}, \mathbf{r} \in B_{2^{4k+10}}$  where

$$B_n \triangleq [n]_0 \times [n]_0$$

Moreover a polychromatic square will be denoted if all the points in the neighbouring squares satisfy the bipolar colouring. Moreover below we will denote our colouring scheme for clarity:

$$\begin{aligned} + &= (+1, +1) \\ \pm &= (+1, -1) \\ \mp &= (-1, +1) \\ - &= (-1, -1) \end{aligned}$$

First, we define mapping  $\mathcal{F} : V_{2^k} \rightarrow B_{2^{4k+10}}$  such that

$$\mathcal{F}(\mathbf{u}) = \begin{pmatrix} 6\mathbf{u}_1 + 6 \\ 6\mathbf{u}_2 + 6 \end{pmatrix} \in B_{2^{4k+10}}$$

Since  $G \in C_{2^k}$ , its edge-set can be decomposed into a collection of paths and cycles  $(P_i)_{i \in [m]}$ . By



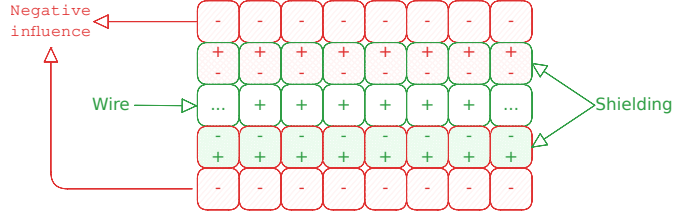


Figure 2.1.6: Shielding method on 2D-EoL embeddings.

using  $\mathcal{F}$ , every  $P_i$  is mapped to a set  $I(P_i) \subset B_{2^{4k+10}}$ . We will colour every point on  $I(P_i)$  with +

$$P = \mathbf{u}^1, \dots, \mathbf{u}^t, \text{ if } P \text{ is a cycle } \mathbf{u}^1 = \mathbf{u}^t \quad (2.1.2)$$

$$I(P) \triangleq \bigcup_{i \in [t-1]} I(\mathbf{u}^i \mathbf{u}^{i+1}) \quad (2.1.3)$$

A lot of *Line reductions* under colouring use the notion of shielding. Essentially we embed our lines and cycles in a grid, everything not a line is denoted as a negative charge  $-$ . All lines are denoted as positive charge  $+$ . To protect our lines from creating solutions, we cover them with  $\pm, \mp$  lines. A visualisation can be seen in figure 2.1.6. We define the following set to denote the set of shielding points around our paths.

$$O(P) = \{\mathbf{p} \in B_{2^{4k+10}} \setminus I(P) : \exists \mathbf{p}' \in I(P), \|\mathbf{p} - \mathbf{p}'\|_\infty = 1\}$$

If  $P$  is a simple path we can decompose  $\{\mathbf{s}_p, \mathbf{e}_p\} \cup L(P) \cup R(P)$  such that:  $\mathbf{s}_p = \mathcal{F}(\mathbf{u}^1) + \Delta + \Delta^\perp$ , where  $\Delta = (\mathbf{u}^1 - \mathbf{u}^2)$ ,  $\mathbf{e}_p = \mathcal{F}(\mathbf{u}^1) + \bar{\Delta} + \bar{\Delta}^\perp$ , where  $\bar{\Delta} = (\mathbf{u}^t - \mathbf{u}^{t-1})$  and  $x^\perp$ , denotes the vector perpendicular to  $x$ . Choice of which perpendicular vector we consider is arbitrary. Starting from  $\mathbf{s}_p$ , enumerate vertices in  $O(P)$  clockwise as  $\mathbf{s}_p, \mathbf{q}^1, \dots, \mathbf{q}^{n_1}, \mathbf{e}_p, \mathbf{r}^1, \dots, \mathbf{r}^{n_2}$  such that

$$\begin{aligned} L(P) &= \{\mathbf{q}^i\}_{i \in [n_1]}, \\ R(P) &= \{\mathbf{r}^i\}_{i \in [n_2]} \end{aligned}$$

If  $P$  is a simple cycle, then can can decompose  $L(P) \cup R(P)$ , where  $L(P)$  contains all the vertices on the left side of the cycle and  $R(P)$  all the vertices on the right side. If  $G$  is specified by  $(K, 0^k) \subset C_{2^k}$ , we can uniquely decompose the edge set into  $P_1, \dots, P_m$ . Every path is either a maximal path, or a cycle in  $G$ . We can use the following to describe our colouring algorithm:

Where  $M_k(\mathbf{p})$  is can be described with the following description:

$$M_k(\mathbf{p}) = \begin{cases} + & \text{if } \exists i, \mathbf{p} \in I(P_i) \\ \pm & \text{if } \exists i, \mathbf{p} \in L(P_i) \vee p_1 = 0 \\ \mp & \text{if } \exists i, \mathbf{p} \in R(P_i) \vee p_2 = 0 \\ - & \text{otherwise} \end{cases}$$

We will call the path of nodes  $(0,0) \rightarrow (0,6) \rightarrow (6,6)$  as the tail, which ensures that the  $0^n$  node of our original graph is not a solution. An example of how such reduction looks like can be seen in figure 2.1.7.

---

**Algorithm 2.1.1** Turing machine  $F$  with input  $(p_1, p_2) \in B_{2^{4k+10}}$

---

```

if  $p_1 = 0$  then
  if  $p_2 \leq 6$  then
    return  $+$ 
  else
    return  $\pm$ 
else if  $p_1 < 6$  then
  if  $p_2 = 6$  then
    return  $+$ 
  else if  $p_2 < 6$  then
    return  $\mp$ 
  else if  $p_2 = 7$  then
    return  $\pm$ 
  else
    return  $-$ 
else
  return  $M_K(p)$ 

```

---

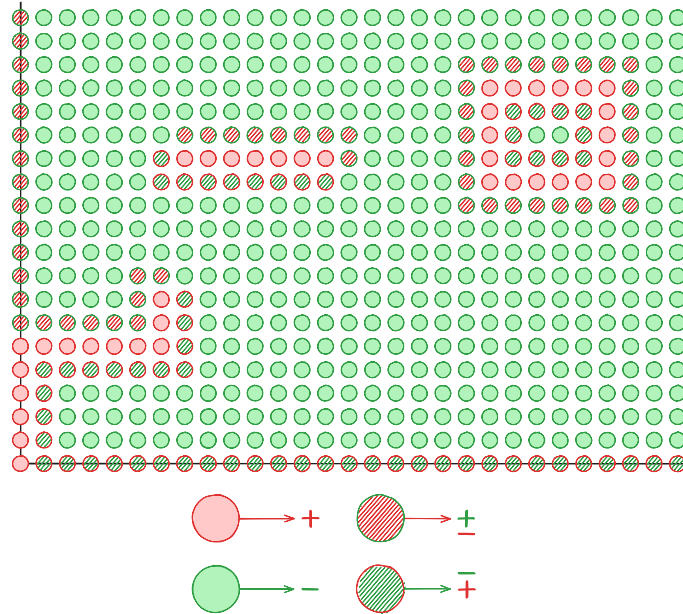


Figure 2.1.7: 2D EndOfLine Bipolar Colouring

**Lemma 2.1.2: Correctness Lemma**

The following statements hold correct:

1.  $\forall i, j : \min_{x \in P_i, y \in P_j} \|\mathcal{F}(x) - \mathcal{F}(y)\|_\infty > 4$
2. All polychromatic squares represent solutions

To prove the first statement, assume we have  $a, b \in V_{2^k} : \|a - b\|_\infty \geq 1$ . WLOG assume  $a - b = (x, y)$  for some  $x, y \in \mathbb{N}$  which implies the following:

$$\mathcal{F}(a) = \mathcal{F} \begin{pmatrix} b_x + x \\ b_y + y \end{pmatrix} = \begin{pmatrix} 6b_x + 6x + 6 \\ 6b_x + 6y + 6 \end{pmatrix}$$

$$\mathcal{F}(a) - \mathcal{F}(b) = \begin{pmatrix} 6x \\ 6y \end{pmatrix}$$

Since  $x + y > 0 \implies \|\mathcal{F}(a) - \mathcal{F}(b)\|_\infty \geq 6 > 4$ . This statement is meant space all lines and their

shieldings are far enough from each other such that the colourings will not create additional solutions. Visually what this looks like is, if we had to parallel lines in  $V_{2^k}$ , then the mappings with the colourings, will look like the figure in 2.1.8.

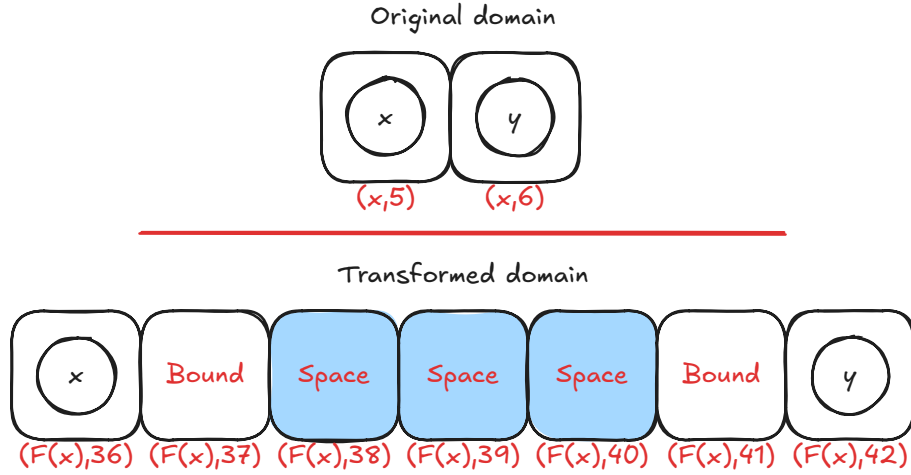


Figure 2.1.8: Transformed domain mapping example.

To show our second part of our solution, we can observe that: All nodes that are not boundary not in  $\bigcup_i I(P_i) \cup L(P_i) \cup S(P_i)$  are coloured  $-$ . All  $+$  coloured nodes are on the tail and in all lines, all non-tail nodes are cushioned by  $\pm, \mp$  and every non-tail node is not poly chromatic. All  $-$  elements do not affect  $\pm, \mp$  coloured nodes. Since all lines are sufficiently apart from each other, it suffices to check only the ends of paths, as for squares in the lines we can observe that we only have labels  $\{\pm, +\}$  or  $\{\mp, +\}$ . One can observe that the square that contains  $s_{P_i}$  and the start of a line, or the square that contains  $e_{P_i}$  and the end of a line will be polychromatic. Therefore, we have a valid and parsimonious reduction since we only cubes and the beginnings or ends of lines can be solutions. We call this coloured instances as *2D-StrongRLeafD*

**Going from 2D to higher dimensions** We now demonstrate how to go from *2D-StrongRLeafD* to *AntipodalStrongSperner*. We employ a slightly adjusted method than one proposed by Chen et al. [8], which closely resembles the *Snake Embedding* technique used by Deligkas et al. [14] on the *StrongTucker* version of the problem, which reduced the space from  $2^n \times 2^n \rightarrow 7^m$ , where  $m = O(n)$ . We will show how to expand their method for the *StrongSperner* problems as well as demonstrate that such reduction will resolve in an *AntipodalStrongSperner* instance.

### Theorem 2.1.2

$$\#\text{PPAD}(\text{2D-STRONGEOL}) \subseteq \#\text{PPAD}(\text{ANTIPODALSTRONGSPERNER})$$

*Proof.* General idea is that we have these twisted sheets that capture the  $n - 1$  embedding onto the higher on We are given as an input a  $2^n \times 2^n$  grid representing any *2D-StrongEoL* problem. The procedure is summarised as follows: Given dimension  $n$ , apply the **padding operation** such that  $n' = 3 \cdot s + 1$  for some  $s \in \mathbb{N}$ . Then apply the *folding* operation, which reduces “width” of the space to  $s + 3$  and compresses the space into a higher dimension. This resembles how we can take a sheet of paper and fold it half way such that it now has a third dimension, but a smaller width on the folded one. We will now go into a greater detail as to how each operation works. To make notation simple and consistent, we define  $M^t$  as

$$M^t \triangleq \bigtimes_{i \in [n_t]} m_i^t$$

Where  $m_i^t$  denotes the width of dimension  $i$  at iteration  $t$ , and  $n_t$  as the number of dimensions.

**Padding Dimensions** We will use the definition 2.1.1 to define our operation.

**Definition 2.1.8: Padding dimension  $P(T, i, u)$**

Given labelling function  $\Lambda^t : M^t \rightarrow \{-1, +1\}^{n_t}$ , we define the padding operation  $P(\Lambda^t, i, u)$  for  $i \in [n_t]$  and  $u \in \mathbb{N}$  where  $n_{t+1} = n_t$  and

$$\forall j \in [n_{t+1}] : m_j = \begin{cases} m_i & \text{if } i \neq j \\ m_i + u & \text{otherwise} \end{cases}$$

The above operation extends dimension  $i$  with  $u$  additional layers. How the operation work is that we append a *null* dimension such that, given  $\bar{x} = x_{-i}(m_i^t + 1)$ :

$$\forall x \in M^t, j \in [n] : [\Lambda^{t+1}(\bar{x})]_j = \begin{cases} -1 & \text{if } \bar{x}_i > 0 \\ +1 & \text{otherwise} \end{cases}$$

We can be sure that the above will not result in any new solutions since in  $[\Lambda^t(\cdot, m_i^t, \cdot)]_i = -1$ , and therefore any solution between  $(\cdot, m_i^t, \cdot), (\cdot, m_i^t + 1, \cdot)$  will not yield in a solution.

**Snake Folding** below we will give a overview of the snake folding operation. Before giving a formal definition, we will first give an informal overview of what the technique actually does based on the figure 2.1.9. The  $y$ -axis denotes the new dimension, denoted as  $d'$  and the  $x$  axis denotes the folded dimension referred as  $\bar{i}$  with width  $m_i^{t+1} = s + 3 = m_i^t$ , given that the original width was  $3s + 1$  for some  $s \in \mathbb{N}$ . Moreover, we will use notation  $(a, b)$  as  $a$ , the labels in the  $n_t$  subspace and  $b$  as the label of the  $n_t + 1$  dimension. From the figure we observe that we have orange, red, green and blue lines. Blue and orange lines denote  $(\omega, -)$  and  $(\omega, +)$  where  $\omega$  is the  $d$ -dim null space, where for each point  $> 0$ , we use label  $-$  and for 0 we use label  $+$ . For the red and green lines, these are denoted as  $(\star, -), (\star, +)$  where  $\star$  is an  $n$ -dim copy of the lower space with some additional points. The core idea is that all solutions, must belong in between the red and green lines, and if we have a solution in the  $n_t$ -space, we should also have a solution in the  $n_t + 1$  space, as shown in the figure 2.1.10.

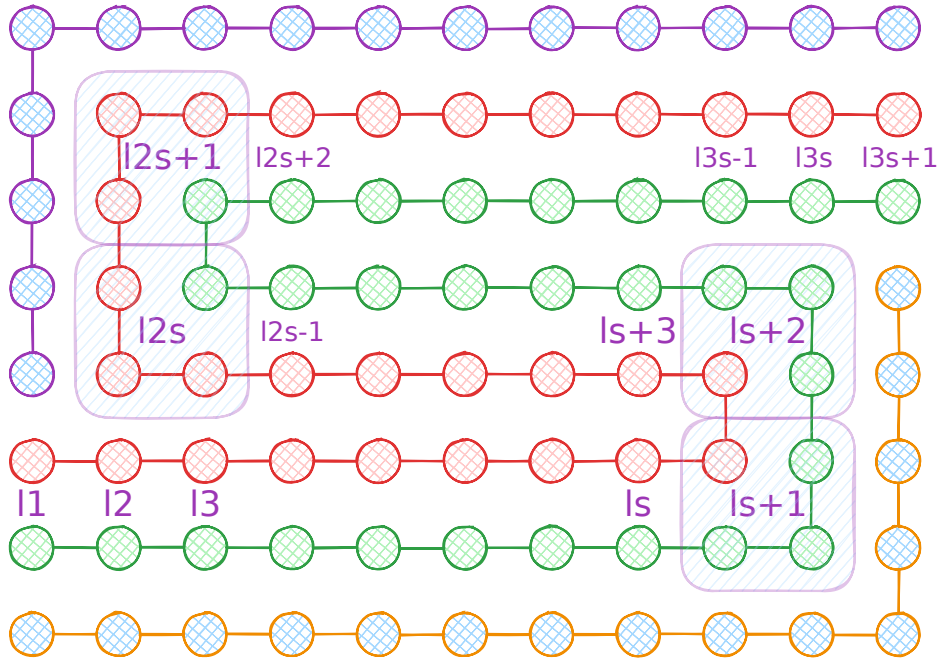


Figure 2.1.9: Snake Embedding technique between the folding dimension and the new dimension

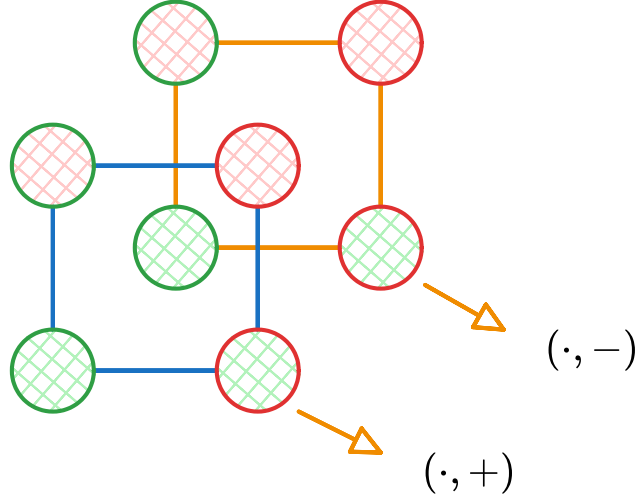


Figure 2.1.10: Snake Embedding technique between the folding dimension and the new dimension

To define the above more formally, we define sets  $B^U, B^L \subseteq M^t$  which denote the set of boundary indices of the orange and blue lines. In addition, we define functions  $\ell^P, \ell^N \subseteq M^t$  to denote the points on the green and red lines with functions  $\phi^P, \phi^N$  that are the inverse maps  $\phi^i : \ell^i \rightarrow M^t$  that indicate the indexes in the original space. Given our new circuit  $\Lambda^{t+1}$ , we define the following:

$$\forall x \in M^{t+1}, j \in [n_t] : [\Lambda^{t+1}(x)]_j = \begin{cases} -1 & \text{if } x \in B^U \cup B^O \wedge x_j > 0 \\ +1 & \text{if } x \in B^U \cup B^O \wedge x_j = 0 \\ [\Lambda^{t+1}(\phi^P(x))]_j & \text{if } x \in \ell^P \\ [\Lambda^{t+1}(\phi^N(x))]_j & \text{if } x \in \ell^N \end{cases}$$

For the new dimension, we apply the labelling rules as described previously

$$\forall x \in M^{t+1} : [\Lambda^{t+1}(x)]_{n_t+1} = \begin{cases} -1 & \text{if } x \in B^U \cup \ell^N \\ +1 & \text{if } x \in B^L \cup \ell^P \end{cases}$$

Using the above we formally define the operation the snake embedding operation  $S(T, i)$ .

**Definition 2.1.9: Snake Embedding operation  $S(\Lambda^t, i)$**

Given labelling function  $\Lambda : M^t \rightarrow \{-1, +1\}^n$ , apply the snake embedding operation on dimension  $i \in [n]$ , where  $m_i \equiv 1 \pmod{3}$ , to get a new labelling function  $\Lambda' : M^{t+1} \rightarrow \{-1, +1\}^{n+1}$  such that  $m'_i = s + 3, n_{t+1} = n_t + 1, m_{n_{t+1}} = 7$ , where  $\Lambda^{t+1}$  is defined based on the above description.

**Claim 2.1.4: Antipodality Solution preservation**

Given operation  $S(\Lambda^t, i)$ , we argue the following hold true:

1. If  $\Lambda^t$  which is antipodal, it will be preserved in the operation.
2. The number of solutions between the two spaces will be preserved

*Proof.* We observe to the labelling conditions by comparing the original with its folded state. Assume we apply the snake folding operation on dimension  $j$ . We can observe that all solutions in the embedded space will be anchored between the red or green lines (using figure 2.1.9),

otherwise the new dimension will not be covered. Moreover, one can observe that a panchromatic hypercube can never be anchored in any of the turning points since our labelling is discrete and we know that any  $n_t - 1$  subspace cannot yield any panchromatic solutions. Assume we have a point in  $p \in M^{t+1}$ , which is an anchor of some panchromatic solution for some  $p_j \in [m_j^{t+1}]$  and  $p_x \in [m_{n_{t+1}}^{t+1}]$ . We can observe from the diagram 2.1.9 that the coordinates  $(\cdot, p_j, p_x)$  correspond to some point  $(\cdot, j)$  for some point in the pre-folded space. WLOG assume that  $(\cdot, j + 1)$  corresponds to the point  $(\cdot, p_j + 1, p_x)$ . That would imply that points  $(\cdot, p_j, p_x), (\cdot, p_j + 1, p_x)$  yield a panchromatic solution for the  $n_t$ -dimensional subspace up to the first  $n_t$ -dimensions. In order to cover the  $n_{t+1}$ -th dimension, we can observe that the points  $(\cdot, p_j, p_x + 1), (\cdot, p_j + 1, p_x + 1)$  with  $(\cdot, p_j, p_x), (\cdot, p_j + 1, p_x)$  are sufficient to cover all labels. But we can observe that those solutions points are part of the  $n_{t+1}$ -dim cube which implies that we have a panchromatic solution. This would also imply that if the  $n_t$  subspace of the cube does not cover the first  $n_t$  labels, then the folded dimension will also not yield any labels, which implies that the number of solutions is preserved between foldings. Moreover we can observe that given our previous definition,

$$\forall j \in [n], b \in \{0, 1\}^n : [\lambda(p + b0)]_j = [\lambda(p + (1^n \oplus b)0)]_j$$

And

$$\forall b \in \{0, 1\}^n : [\lambda(p + b1)]_j = -[\lambda(p + (1^n \oplus b)1)]_j$$

Which implies that our cube is also antipodal □

We observed that our operations do not affect the cardinality of solutions. We now outline our construction of our new circuit

1. Check if there is a dimension with  $m_i^t > 8$ . If yes, first pad the dimension such that  $m_j^t \equiv 1 \pmod 3$ . Afterwards fold the dimension on  $j$  to get new folded circuit
2. If the folding results in a width of  $m_i^t < 8$ , then we can pad the dimension to have a width of 8
3. If no operations were done then we can terminate

We can observe that in each step of the algorithm we preserve our solution. Moreover, each fold divides the dimension by some factor of 3. We can therefore observe that the runtime for the creation of such circuit will take  $O(n \log_3 2)$ . This concludes our proof. □

This is the closest that we manage to bring the reduction chain across *EndOfLine* and *PureCircuit*. We argue the antipodality property might offer useful properties as we can immediately discern solutions by looking at antipodal pairs as it is much more computationally efficient than trying to find  $d + 1$  points in some hypercube and it doesn't scale with the dimensionality of the space. Unfortunately, we haven't figured a way to exploit, this property with hazard-free logic and we leave it as a future direction.

Lastly, we observe that throughout literature such as [20] and [15], both constructions take advantage of speculative computing, where we calculate the input on multiple copies of the value and use the result of the majority. Unfortunately, while this can resolve a lot of the reductions, it may lead to 1-to-many mapping between the solutions of the original problem domain and the *PureCircuit* domain.

### 2.1.2 SourceOrExcess analysis

The current section focuses on how we can extend our reductions to *SourceOrExcess*. In the previous sections, we gave an overview on how we can go from an *EndOfLine* problem to



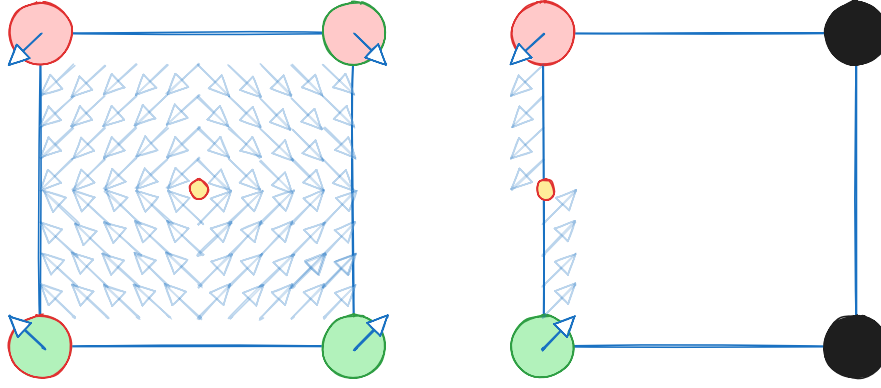


Figure 2.1.11: Panchromatic edge for 2 dimensions

a topological problem. In the current section, we show how we can extend the idea for the *SourceOrExcess* problem. We hope by parsimoniously reducing the *SourceOrExcess* problem to a topological problem, it will lead to future reductions to the *PureCircuit*.

### Topological representations

#### Topological Parsimonious Reduction

In section , when talking about the *StrongSperner* problem, we observed how coloured subspaces may lead to many duplicated solutions. Here, we are slightly modifying the definition of *3D-StrongSperner* in order to take advantage of that. We will then show how the above modification can lead to parsimonious reductions.

#### Definition 2.1.10: 3D-CountingSS problem definition

**Input:** A hazard-free colouring circuit  $\lambda : (\{0, 1\}^k)^3 \rightarrow \{-1, +1\}^3$  which follows the boundary conditions of the *StrongSperner* problem as explained in 1.2.13.

**Output:** A coordinate  $p = (ib_1, jb_2, kb_3) \in (\{0, 1, \perp\}^k)^3$  such that  $\lambda(p) = \perp^3$  for some  $b \in \{0, 1, \perp\}^3$ . To avoid double counting, accept only solutions such that

$$\max\{(ib_1, jb_2, kb_3) \mid b \in \{0, \perp\}^3 : \lambda(ib_1, jb_2, kb_3) = \perp^3\}$$

Where the max follows the information ordering of Kleene logic.

The above idea is a natural extension of the *StrongSperner* interpretation, where its labelling represents the direction of a continuous function. If a face of a cube is panchromatic, then that implies that there has to be a fixed point somewhere, as seen in the figure 2.1.11. If an edge were to be panchromatic, then that means that a fixed point could lie on the edge. If we were treating this as a fixed point problem instead of a colouring one, it would make sense to argue that we have one solution. We abuse this notion to count only the smallest possible subspace that contains the solution, for example in 2D, only count the edge that yield the labelling instead of all the cubes that contains the edge.

To prove **PPAD**-completeness, we know that the above problem can be reduced to *3D-StrongSperner*, although the reduction will not be parsimonious. We now show the main result of our section:

#### Theorem 2.1.3: Parsimonious reduction from *SourceOrExcess*(2, 1)

$$\#\text{PPAD}(\text{SourceOrExcess}(2, 1)) \subseteq \#\text{PPAD}(3\text{D-CountingSS})$$

Given a *SourceOrExcess* input, we apply the following series of transformations:

1. **Circuit Augmentation:** Eliminate all excess nodes such that the graphs only contain sources, sinks, isolated nodes, line nodes and double sinks.
2. **Planar Embedding:** Embed the graph in a 3D coordinate space.
3. **Topological colouring:** Apply colouring rules that follow the *3D-CountingSS* rules such that our panchromatic squares contains *SourceOrExcess* solutions.

**Circuit Augmentation** We attempt to eliminate all excess nodes, meaning nodes with degree (2, 1) by separating one of the lines as shown in the figure 2.1.12. Moreover for double sinks, we enforce that the MSB value of the bit-string for one of the two values is one. We can observe that these modifications can be done in polynomial time since they only require to analyse the local neighbouring of nodes, and they preserve the number of solutions. We emphasize that if a node is not incident to double sink, then it will act as an isolated node.

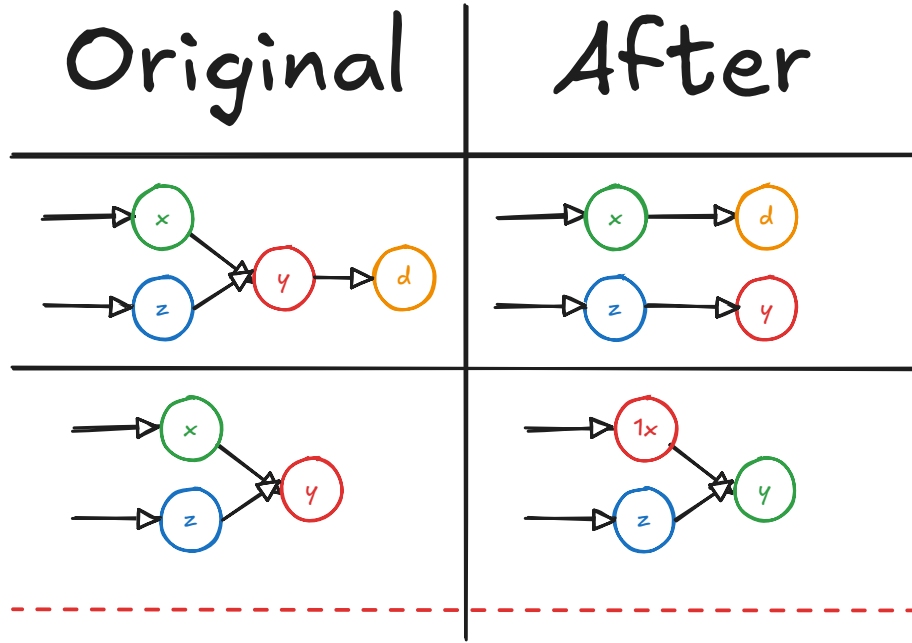


Figure 2.1.12: The current table shows how we deal with excess nodes. All other nodes such as sources, sinks, lines or isolated vertices stay the same.

**Planar Transformation** We will propose a similar transformation to the 1.2.1 to encode a 2D representation of the above graph. Given  $n$ , we create a 2D space  $B_n$  such that

$$V_n \triangleq \{p \in \mathbb{Z}^2 \mid 0 \leq p_1 \leq 6n^2, 0 \leq p_2 \leq 12n + 6\}$$

We are essentially increasing the dimensions of our cube so we can increase the distance between points. For every  $i \in \{0, 1\}^{n+1}$ , vertex  $i$  represents point  $(0, 12i)$  Like in the original definition we also create paths between  $E_{ij}$  but we for the double sinks we create a different path. More



accurately,  $\forall i, j \in E_n$  such that  $i^{n+1} = 0$ , create path

$$\begin{aligned} \mathbf{p}^1 &= (0, 12i) \\ \mathbf{p}^2 &= (6n(ni + j), 12i) \\ \mathbf{p}^3 &= (6n(ni + j), 12j + 6) \\ \mathbf{p}^4 &= (0, 12j + 6) \\ \mathbf{p}^5 &= (0, 12j) \\ E_{ij} &\triangleq E(\mathbf{p}^1 \mathbf{p}^2) \cup \dots \cup E(\mathbf{p}^4 \mathbf{p}^5) \end{aligned}$$

If  $j$  is a double sink, attach a small tail:

$$\begin{aligned} \mathbf{p}^6 &= (3, 12j) \\ E'_{ij} &\triangleq E(\mathbf{p}^5 \mathbf{p}^6) \end{aligned}$$

If  $i^{n+1} = 1$ , then we create path

$$\begin{aligned} \mathbf{p}^1 &= (0, 12i) \\ \mathbf{p}^2 &= (6n(ni + j), 12i) \\ \mathbf{p}^3 &= (6n(ni + j), 12j) \\ \mathbf{p}^4 &= (3, 12j) \\ E_{ij} &\triangleq E(\mathbf{p}^1 \mathbf{p}^2) \cup E(\mathbf{p}^3 \mathbf{p}^4) \cup E(\mathbf{p}^3 \mathbf{p}^4) \end{aligned}$$

In the original problem, we argued that for every node, we have I/O lines that denote the input output nodes. If a node has in-degree of 2, then there must exist a node of in-degree 2 out-degree 1 as seen in the figure . This would make it computationally difficult to differentiate between double sinks as a line would be exponentially long and for each double sink node, and we would end up double counting solutions. By removing excess nodes, we only have to focus on double sinks. Since there is no output line in a double sink, we take advantage of that by sending one input from the traditional input line, and the other from the output line.

We can use similar arguments as in the original proof, to argue that:  $\forall v \in V : \text{degree}(v) \in \{(0, 1), (1, 0), (1, 1), (2, 2), (2, 0)\}$ . We can observe that, if we do not have any double sinks, our graph follows the original transformation where only 4 out of the 5 possible degrees are possible. For the  $(2, 0)$  case, using figure as reference 1.2.3, it suffices to observe what happens in the double sink case. We can observe that the tail of the  $E'_{ij}$  path finishes at  $(3, 12j)$  . The smallest possible height for a transition line would be at a height of  $(6n(ni + j))$ , which means the smallest possible height between two nodes is 6. Meaning all nodes at  $(3, 12j)$  do not intersect any other path other than their double sink counterparts. This concludes our argument.

We follow the same construction for the  $E_n^2$  and apply the same removal transformation. The resulting graph is the same as in the traditional *RLeafD* problem but with the addition of double sinks.

**Topological colouring** This section we show how to colour our circuit to on a *3D-CountingSS* instance. First we apply the transformation  $\mathcal{F}$  to go from a *2D* space into a *3D*, where

$$B_n \triangleq \{\mathbf{p} \in \mathbb{N}^3 \mid p_1 \leq 36n^2 + 6, p_2 \leq 78n + 42, p_3 \leq 6, \}$$

$$\mathcal{F} : V_n \rightarrow B_n$$

$$\forall \mathbf{p} \in V_n : \mathcal{F}(\mathbf{p}) \triangleq \begin{pmatrix} 6 \cdot p_1 + 6 \\ 6 \cdot p_2 + 6 \\ 0 \end{pmatrix}$$

This mapping essentially re-creates the *RLeafD* problem in a *3D* space, since we are neglecting the 3rd dimension for the most part. Like in the original transformation, we can decompose our graph into paths, cycles where  $(P_i)_{i \in [m]}$ . By using  $\mathcal{F}$ , every  $P_i$  is mapped to a set  $I(P_i) \subset B_n$ , such that

$$P = (\mathbf{u}^1, \dots, \mathbf{u}^t), \text{ if } P \text{ is a cycle } \mathbf{u}^1 = \mathbf{u}^t$$

$$I(P) \triangleq \bigcup_{i \in [t-1]} I(\mathbf{u}^{t-1}, \mathbf{u}^t)$$

For double sinks, we can treat them as two paths that lead to a point, we can therefore decompose double sinks as  $I(P_i) \cup I(P_j) \cup \{\mathbf{m}^{ij}\}$ , where  $\mathbf{m}^{ij}$  will be denoted as the meet point of the two paths.

Our shielding notion for this tranformation will work follows. We again define set points around each line  $O(P)$  as such that

$$O(P) \triangleq \{\mathbf{p} \in B_n \setminus I(P) \mid \exists \mathbf{p}' \in I(P) : \|\mathbf{p} - \mathbf{p}'\|_\infty = 1\}$$

$$\forall b \in \{0, 1\} : O^b(P) \triangleq \{\mathbf{p} \in O(P) \mid p_3 = b\}$$

If  $P$  is a simple path, then we can decompose into  $\{\mathbf{s}_p, \mathbf{e}_p\} \cup L^0(P) \cup R^0(P)$  such that  $\mathbf{s}_p = \mathcal{F}(\mathbf{u}^1) + \Delta + \Delta^\perp$  and  $\mathbf{e}_p = \mathcal{F}(\mathbf{u}^t) + \bar{\Delta} + \bar{\Delta}^\perp$  where:  $\Delta = (\mathbf{u}^1 - \mathbf{u}^2), \bar{\Delta} = (\mathbf{u}^t - \mathbf{u}^{t-1})$  and  $\mathbf{x}^\perp$  denotes the perpendicular vector when rotated by  $90^\circ$  anticlockwise across the  $xy$  plane. For simple paths and cycles, the decomposition into  $L$  and  $R$  sets works the same, with the core difference being that we have two sets of points, one for  $z = 0$  and the other for  $z = 1$ . For double sinks, assume that paths  $P^1, P_2$  intresect at some pont  $\mathbf{m}^{ij}$ . We apply the following partition starting from  $\mathbf{s}_{p1}$

$$O^0(P_i \oplus P_j) = \{\mathbf{s}_{p1}, \mathbf{u}_1^1, \mathbf{u}_2^1, \dots, \mathbf{u}_k^1, \mathbf{e}_{p1}\} \text{ Left of first path up to interection point}$$

$$\cup \{\mathbf{r}_1^2, \mathbf{r}_2^2, \dots, \mathbf{r}_k^2, \mathbf{s}_{p2}\} \text{ Right of path 2}$$

$$\cup \{\mathbf{u}_1^2, \mathbf{u}_2^2, \dots, \mathbf{u}_k^2, \mathbf{e}_{p2}\} \text{ Left of path 2 up to interection}$$

$$\cup \{\mathbf{r}_1^1, \mathbf{r}_2^1, \dots, \mathbf{r}_k^1, \mathbf{s}_{p2}\} \text{ Right of path 1}$$

We ensure that  $\mathbf{e}^1, \mathbf{e}^2, \mathbf{m}^{ij}$  are on the same  $x$ -coord or  $y$ -coordinate. The same sets can be done for  $O^1(P_i \oplus P_j)$ . We now propose our colouring circuit ( $\mathcal{M}$ ) such that

$$\mathcal{M}(\mathbf{p}) \triangleq \begin{cases} (+, +, (-1)^{p_3}) & \text{if } \exists i, \mathbf{p} \in I(P_i) \vee \mathbf{p} \in \{(0, 0, 0), (0, 0, 1)\} \\ (-, +, (-1)^{p_3}) & \text{if } \exists i, \mathbf{p} \in R(P_i) \\ (+, -, (-1)^{p_3}) & \text{if } \exists i, \mathbf{p} \in L(P_i) \\ (-, -, (-1)^{p_3}) & \text{if } \exists ij, \mathbf{p} = \mathbf{m}^{ij} \\ (-, -, (-1)^{p_3}) & \text{otherwise} \end{cases}$$

An example of how the above colouring for simple lines can be seen in figure 2.1.13.

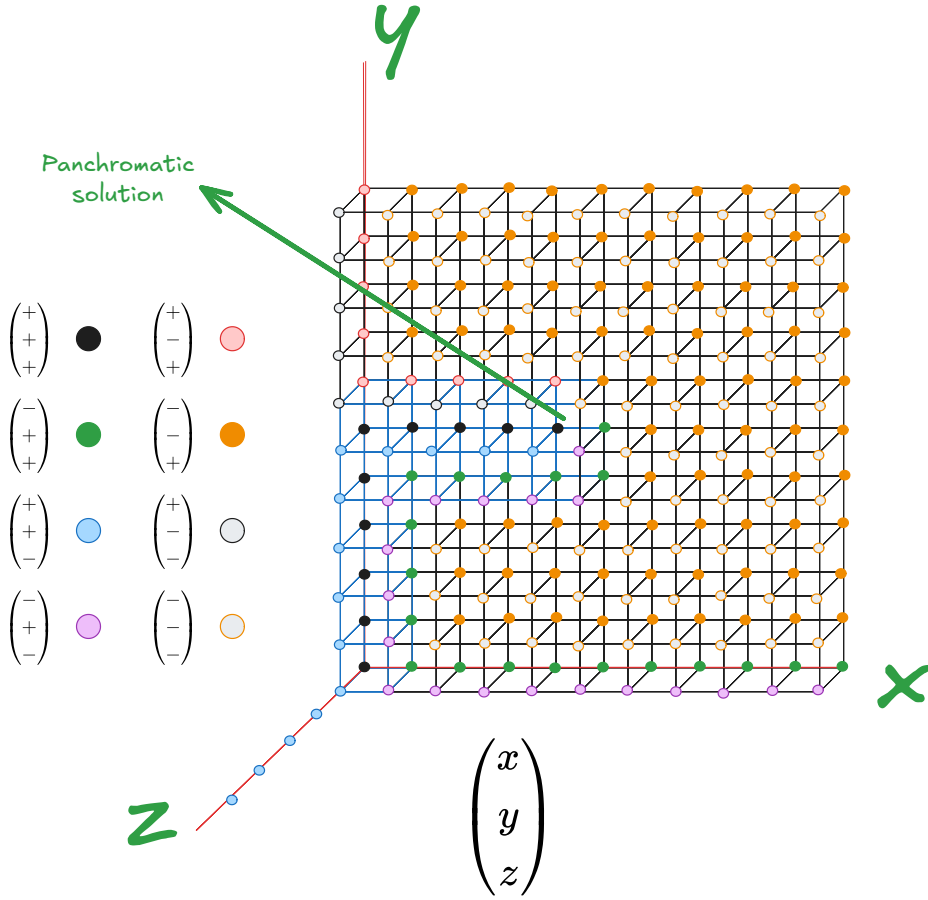


Figure 2.1.13: Colouring example based on the proposed circuit

**Correctness proof** We now prove that our reduction is correct. We use figure 2.1.14 and assume that path  $\mathbf{p}^t, \mathbf{p}^{t+1}$  is in the middle of some line. The red cube will not be panchromatic as  $y$ -coordinate is  $+$  on all corners and vice versa for the blue cube on the  $x$ -coordinate. In addition we can observe that the brown cubes surrounding each face will also not result in a panchromatic cube as: for the brown cube on the left of red will be  $-$  on all  $y$  coordinates, with the same argument for blue. And for the brown cubs on top, we can observe that  $z$ -coordinate will be  $-$  on all points. Therefore points in between lines or cycles will not result in solutions.

As we can observe from the figure 2.1.13, we can see that in the ends of lines, we can use the colouring as shown in the figure. We can observe that in our figure, the corner with the max coordinates and the one with min coordinate cover all the labels. In addition we can observe that each side of the cube will not cover all the labels therefore there are no overlapping panchromatic cubes.

For the double sink case, we use the figure 2.1.15, to observe what happens around the double sink area. For points  $m_{-3}^{ij}(0), m_{-3}^{ij}(1)$ , we can observe that their labellig is antipodal, meaning given our counting defintion, all surrounding cubes that contain the side  $(m_{-3}^{ij}(0), m_{-3}^{ij}(1))$  count as one solution. It suffices to observe that all faces of the current section are not panchromatic, which implies other than the 4 cubes which we account 1, there are no other additional solutions. Therefore we can observe that the set of panchromatic solutions correspond to a 1-to-1 mapping with the *SourceOrExcess* problem.

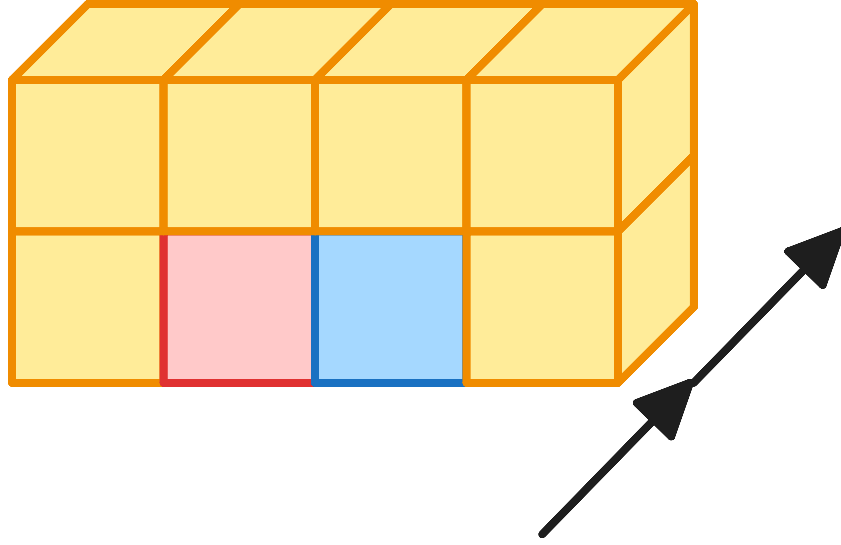


Figure 2.1.14: Example colouring where the arrow denotes the direction. In between the red and blue cubes in the bottom corners we have the positive line. The brown cubes around denote the negative labels

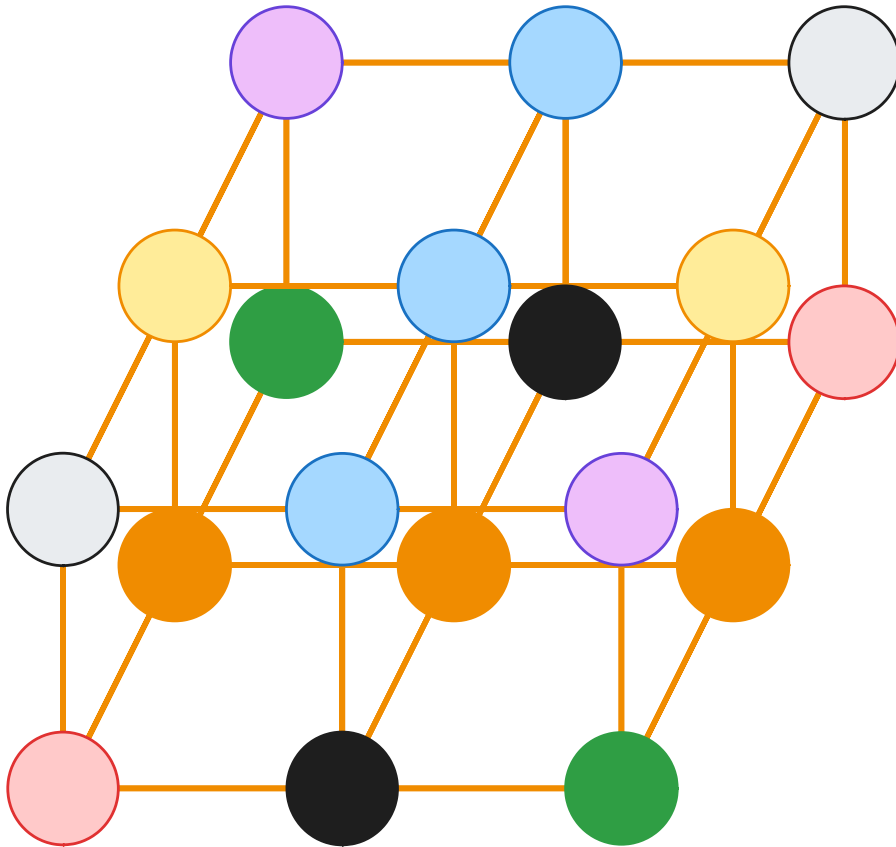


Figure 2.1.15: Colouring around the double sink points. The colours correspond to the colour legend of 2.1.13

**Future Directions** From the previous section we observed how to reduce  $SourceOrExcess(2, 1)$  to a topological problem. Just like in the *EndOfLine* problem, we found it essential to reduce it to a *StrongSperner* problem with polynomial sides and very high dimensionality. We believe that one can abuse the *Snake Embedding* methodology that we proposed earlier. Indeed one can observe that with correct foldings, the  $n$ -dimensional space will contain a  $n - 2$  panchromatic subspace.

There are two main questions that need to be resolved:

1. Will the number of solutions between foldings?
2. Is there a natural way to define *Counting-SS*? Like previously, when the number of dimensions can vary between instances, by corollary 1.2.3, we observed that the size of the circuit will explode exponentially, by current literature assumptions.

More interestingly, in later sections we make references of  $\alpha - SUnbalanced(\Delta_i, \Delta_o)$  and how they can affect the counting nature of each other. Like we mentioned previously, for *2D-StrongSperner* and *1-MC*, we cannot guarantee parsimonious reductions, but for a 3D space that was indeed possible. Moreover, Hollender et al. [19] demonstrated that known source variants of the problem do not affect its **PPAD**-completeness. We therefore make the following conjecture:

**Conjecture 2.1.1:  $nD$ -CountingSS and Unbalanced**

Does there exist some polynomial computable function  $f : \mathbb{N}^3 \rightarrow \mathbb{N}$ , such that given  $\alpha, \Delta_i, \Delta_o \in \mathbb{N}$  and  $m = f(\alpha, \Delta_i, \Delta_o)$

$$\mathbf{PPAD}(\alpha\text{-}SUnbalanced(\Delta_i, \Delta_o)) \subseteq \mathbf{PPAD}(mD\text{-}CountingSS)$$

What the above conjecture essentially describes is, given a number of known sources and the maximum number of input/output vertices of graph, can we create some  $nD$ -CountingSS that is parsimonious? This would allow us to bridge graph representations with topological representations and hopefully allow us to get closer to a *PureCircuit* total reduction.

### 2.1.3 PPAD Counting Hierarchies

As previously mentioned, we want to extend the work done by Ikenmeyer et al. [23], in order to analyse the combinatorial nature of **PPAD** as a whole. Although the project did not manage to achieve the desired theorem, we hope to demonstrate interesting findings that can allow us to reveal more intricate properties of **#PPAD**. Our first step is to extend the hierarchy between *EndOfLine* and *SourceOrExcess(2, 1)*. To start we introduce the following fact from Hollender et al. [19]

**Definition 2.1.11: PolyS-EoL definition**

**Input:** We are given triplet  $(S, P, \tilde{f})$ , where  $S, P$  are successors, predecessor circuits of an *EndOfLine* problem with  $n$  input-output bits, and  $\tilde{f}$  is a poly-sized circuit that computes some polynomial function  $f : \mathbb{N} \rightarrow \mathbb{N}$ . In addition, we impose the following restriction:

$$\forall i \in [f(n)]_0 : S(P(i)) \neq i \wedge P(S(i)) = i$$

**Output:** A node  $k \notin [f(n)]_0$  such that  $S(P(x)) \neq x \oplus P(S(x)) \neq x$ .

**Theorem 2.1.4**

*PolyS-EoL* is **PPAD**-complete [19].

We give a formal definition of the *Unbalanced* problem, which is generalisation of the *EndOfLine* to graphs with polynomial sized in- and out-degree.

**Definition 2.1.12: Unbalanced problem definition**

**Input:** We are given tuple  $(\mathbf{S}, \mathbf{P}, \bar{f}, \bar{g})$ , such that  $\mathbf{S} = \{S_i\}_{i \in [f(n)]}$  and  $\mathbf{P} = \{P_i\}_{i \in [g(n)]}$  and  $f, g : \mathbb{N} \rightarrow \mathbb{N}$  are poly-time computable polynomial functions. This describes a graph  $G = (V, E)$  such that  $V = \{0, 1\}^n$  and

$$\forall (x, y) \in E, \exists i \in [f(n)], j \in [g(n)] : S_i(x) = P_j(y)$$

We enforce the condition that  $\forall P_i : P_i(0^n) = 0^n$  and  $\exists S_j : S_j(0^n) \neq 0^n$

**Output:** A node  $v \in \{0, 1\}^n \setminus \{0^n\}$  such that  $\text{in-deg}(v) \neq \text{out-deg}(v)$ .

If  $f(\cdot) = p$  or  $g(\cdot) = k$  for some  $p, k \in \mathbb{N}$ , we will denote the problem as  $\text{Unbalanced}(p, k)$

**Lemma 2.1.3**

$\text{Unbalanced}$  is **PPAD**-complete

*Proof.* The **PPAD**-hardness is trivial since every *EndOfLine* problem is an instance of unbalanced. To show **PPAD**-completeness we demonstrate reduction to *PolyS-EoL*. We are given an unbalanced instance  $(\mathbf{S}, \mathbf{P}, \bar{f}, \bar{g})$ . For each node, we have a predecessor set  $\{p_i\}_{i \in [d_i]}$  and a successor set  $\{s_i\}_{i \in [d_o]}$ , where  $d_i$  and  $d_o$  denote the input and output degrees respectively. We assume the successor-predecessor list is sorted and WLOG and assume we have node  $v \in \{0, 1\}^n \setminus \{0^n\}$  such that  $d_i = d_o + k$ . We create  $d_i$  copies of  $v$  such that: For each  $j \in [d_o]$ , we create edges  $s_j \rightarrow v_j \rightarrow p_j$ . Moreover, we create additional sinks such that  $\forall j \in [k] : s_{d_o+j} \rightarrow v_{d_o+j}$ . We can make a similar argument for  $d_o > d_i$ . We can observe that if  $d_i = d_o$ , then we create no additional solutions. Lastly for the  $0^n$  node, we observe that our construction can result in at most  $g(n)$  additional sources and therefore, we can create an instance of *PolyS-EoL*  $\square$

We can also prove that the number of known sources does not affect the complexity of the *Unbalanced* problem

**Lemma 2.1.4**

$$\#\text{PPAD}(\text{PolyS-Unbalanced}) \subseteq \#\text{PPAD}(\text{Unbalanced})$$

*Proof.* To show our reduction, that we have an instance of *PolyS-Unbalanced*, where have  $f(n)$  known sources. Assume that  $g(n)$  describes the input degree and  $h(n)$  the output degree. We create a new *Unbalanced* instance with in- and out-degree polynomials  $p(n) = (\max\{g(n), h(n)\}) \cdot f(n)$ . We will use the figure .. to illustrate our reduction. Essentially, we are creating  $f(n)$  groups where each group  $i$  contains  $\text{out-degree}(s_i) \leq h(n)$  nodes where  $s_i$  is the  $i$ th known source of the original problem. For each node  $k$  of the  $i$ th group, denoted as  $v_k^i$ , we are adding edge  $v_k^i \rightarrow s_i$ . Since each  $s_i$  element has  $\text{in-degree} = \text{out-degree}$ , we can observe that we are not creating any additional solutions. Since this transformation is local, any other unbalanced node will stay the same and, therefore we can map the two solution sets.  $\square$

Lastly we can make the observation that we can create a 3D-matrix of problems such that

$$\forall i, j, k \in \mathbb{N} \cup \{\omega\} : M_{i,j,k} = kS\text{-Unbalanced}(i, j)$$

Where  $\omega$  denotes polynomial values. The reason why we make this observation is for 3 main reasons. First, this gives us greater insights as to how problems in *PPAD* relate with each other. For example, *EndOfLine* and *2S-EoL*, even though they are very similar and we can reduce directly from one to another, their reductions between each other are non-parsimonious or if there exists one it must be non-trivial. This indicates possibilities for multiple hierarchies with oracle separations of another.

Second reason would be to demonstrate that an important milestone for our conjecture is to establish the relation between *Unbalanced* and *PureCircuit*. By our lemmas 2.1.3, 2.1.3, we can observe that every possible variant of input, output degree and known sources can be eventually reduced to *Unbalanced*. In addition, the majority of problems in *PPAD* are related with each other by some form of an unbalanced directed graph, due to their eventual reductions to the *EndOfLine* problems. We emphasize that this does not imply that every **PPAD** problem can be parsimoniously reduced to the *Unbalanced* problem, but more to indicate some heuristical argument as to why this might be an important problem to study.

Third reason, is that these hierarchies seem to reveal some hidden interesting structures in **PPAD** and in **TFNP** as a whole. As Hollender et al. [19] also indicated, we can observe that in *kS-EoL*, we have at least  $k$  solutions. In fact, we can make additional observations such as, given  $\alpha S\text{-SourceOrExcess}(\beta, 1)$ , by the pigeonhole principle we have at least  $\lceil \alpha/\beta \rceil$  solutions. This implies there exists subclasses in **PPAD** which we can denote as **PPAD**( $k$ ) where we have **PPAD** problems with at least  $k$  solutions. This may lead in future directions where we can ask about combinatorial interpretations of **PPAD**( $\alpha$ ) –  $\beta$  for  $\alpha \geq \beta$ .

## 2.1.4 Supplementary Findings

In this section we would like to present additional findings that do not align with any of our current objectives, but they give insights as to how *PureCircuit* works.

### Hazard Circuits and Tarski

We define the following subclass of **TARSKI** 1.2.17:

#### Definition 2.1.13: KLEENETARSKI problem definition

Given  $F : \mathbb{T}^n \rightarrow \mathbb{T}^n$ , where  $F$  is a *natural* function 1.2.23, find  $x \in \mathbb{T}^n$  such that  $F(x) = x$ . We assume  $F$  will be represented as a circuit that uses set of gates  $\{\wedge, \vee, \neg, 0, 1\}$ .

#### Proposition 2.1.1: Parsimonious Reduction between KLEENETARSKI and PURECIRCUIT

$$\#KLEENETARSKI \subseteq \#PURECIRCUIT$$

*Proof.* Given an input circuit  $C : \mathbb{T}^n \rightarrow \mathbb{T}^n$ , construct **PURECIRCUIT** instance:

1. Initiate a vector of  $n$  nodes, which we denote as  $s$ .
2. Construct the  $C$  using the standard set of gates and apply  $C(x)$  to create output vector  $o$ .
3. Copy the results back into  $s$ .

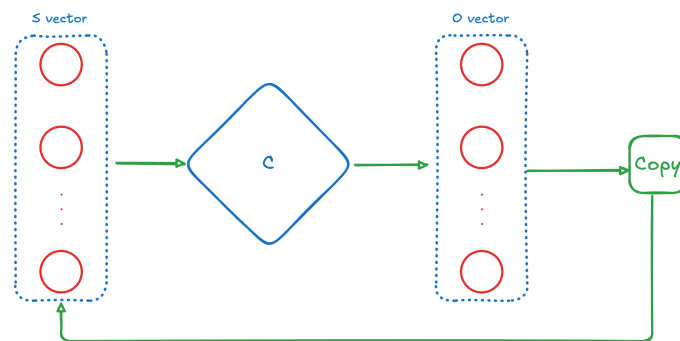


Figure 2.1.16: TARSKI construction

We can visualise the above construction in figure 2.1.16. We can observe that, for any valid assignment  $x \implies x[s] = x[o]$ , which implies  $x[o] = F(x) = x[s]$ .  $\square$

## Reductions to the EndOfLine

One can also attempt to make some reductions to the ENDOfLINE. It has to be noted, that we assume that there are no parsimonious reductions to the ENDOfLINE, but we were able to construct some variants that allows to make there reductions. They key insights for this comes from the PPAD-inclusion proof by Deligkas et al. [15], where they showed that we can reduce PureCircuit, to *Brouwer* problem by constructing a continuous function  $F : [0, 1]^n \rightarrow [0, 1]^n$  as such: For each  $v \in V$ , we create continuous functions  $f_i(\cdot) : [0, 1]^n \rightarrow [0, 1]$  where we take thne input of vector of all the nodes and output the value of the current node

1. If  $v \in V$  is the output of a NOR gate with inputs  $x_1, x_2$ , then we have:

$$f_v(\mathbf{x}) := (1 - \mathbf{x}_{x_1}) \cdot (1 - \mathbf{x}_{x_2})$$

2. If  $y_1, y_2$  are the outputs of a PURIFY gate with  $z$  as input, then we have:

$$\begin{aligned} f_{y_1}(\mathbf{x}) &:= \max\{2 \cdot \mathbf{x}_z - 1, 0\} \\ f_{y_2}(\mathbf{x}) &:= \min\{2 \cdot \mathbf{x}_z, 1\} \end{aligned}$$

If  $F(x) = x$ , then we also found a solution for PURECIRCUIT by:

$$\forall v \in V : \mathbf{x}[v] = \begin{cases} 0 & \mathbf{x}[v] = 0 \\ \perp & 0 < \mathbf{x}[v] < 1 \\ 1 & \mathbf{x}[v] = 1 \end{cases}$$

Given that as an assumption we can restrict our *Purify* gate to outputs  $(0, \perp), (0, 1), (\perp, 1)$ . Using that we can define two types of variants of PURECIRCUIT that are parsimoniously reducible to SOURCEORPRESINK. We first define SOURCEORPRESINK as:

**Definition 2.1.14: SOURCEORPRESINK definition**

A SOURCEORPRESINK is defined by the same construction as the ENDOfLINE but the solutions are sources and predecessors of sinks. If a node is both a source and a presink, then we count it once.

We will annotate our *Purify* gates with an additional bit  $b \in \mathbb{B}$  such that

$$\forall x \in \mathbb{T}, y_1, y_2 \in \mathbb{T} : \text{PURIFY}^b(x) = (y_1, y_2) \implies \text{PURIFY}^{-b}(x) = (y_2, y_1)$$

We can create two types of problems based on the annotation:

1. Acyclic PURECIRCUIT: Our instance is acyclic
2. Permutation-free PURECIRCUIT: The outputs of a pure circuit, are connected to a circuit such that any permutation of the inputs retains the result. More formally:

$$\forall x \in \mathbb{T}^n, \sigma \in \mathcal{A} : C(\sigma(x)) = C(x)$$

Where  $\mathcal{A}$  is the set of automorphisms of  $[n] \rightarrow [n]$ . The above description can be depicted in the following figure: (ADD FIGURE)

We can show that above variants we can create the following proposition (ref). To prove the reduction we can need to argue that we can go from one solution to another. The core idea of these problems is: if we find a solution given in a set of configurations  $\gamma \in \{0, 1\}^p$  where  $p$  are the number of purify gates, we can match it with an additional solution where our set of



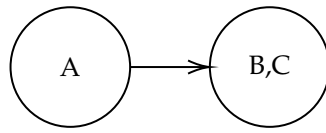


Figure 2.1.17: Equal solution conversion

configurations is  $\neg\gamma$ . This can be visualised as such: If the flip does not affect the inputs then we can create the following graph:

Otherwise for two distinct solutions we can use the following

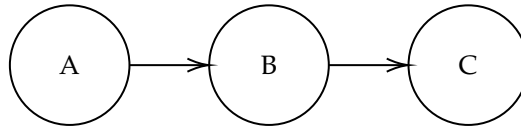


Figure 2.1.18: Not equal solution

## 2.2 Software Deliverable

The current project includes a software element, in order to aid with the gadget development. We provide three core requirements as enumerated in the following list:

1. Visualise and instance of a pure circuit and check for correctness.
2. Given a *PureCircuit* graph, find a satisfying assignment.
3. Given a *PureCircuit* graph, enumerate through all possible assignments.

We split these section into the following components: Tooling selection and development architecture, Visualisation tooling, Solution Finding and Solution Enumeration.

### 2.2.1 Tooling Selection

Using figure .. as reference, we outline the main choice for our tools

**Rust Programming Language** The development language used for is Rust. Rust is a systems-level compiled programming language that can achieve high speeds due to its low-level compilation, while maintaining memory safety. The key reasons, for the specific language of choice is its speed as well as having an advanced algebraic type system.

**Bevy and the ECS architecture** Bevy is a Rust library, primarily used for game development. It uses the Entity-Component-System architecture where entities with varying components exist in world state. Each *entity* interacts with each other based on systems. A visual reference is provided in the figure 2.2.1. The core

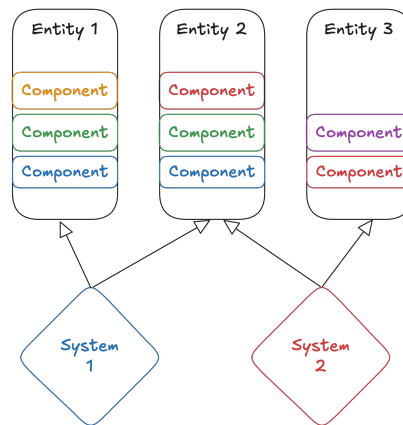


Figure 2.2.1: ECS visualisation implementation

This choice offered several pros and cons. From the one hand, the current choice offers flexibility when it comes to drawing and interacting with shapes on a screen. We wanted to give our tool and editor-like vibe where the user can move the screen and individual nodes themselves as well as interact with them. On the other hand, this library is still in its early stages, which implies limited documentation and features. Moreover, as its prioritizes game development, it has a limitation when dealing with UI concepts and state management. Ultimately, we chose the tool as the UI is simple enough to not hinder the experience of the tool.

---

**Software Testing methodology** The core philosophy of our testing is Test-Driven-Development. This methodology was used to develop and validate our critical parts of our code, such as graph validation or solution enumeration. The main idea of this development is to first write the unit test, and then develop the code to resolve them. This allows us for the code to be self-documenting as well as ensuring that we are acting within the expected scope.

Another testing technique that we heavily took advantage of is *Property-based Testing*, which we implement using the Proptest library <sup>1</sup>. This idea was inspired by the Haskell library QuickCheck [9], where we test the properties of a pure function across a range of inputs. The idea is, the more complex, the input of the function, the greater is the input space and therefore harder and cumbersome to check all possible combinations or edge-cases. With Proptest, we check whether a property of our function is maintained across all possible inputs, such as a sorted output, or a validating assignment. It works by testing our function, across a series of possible inputs. If a failed condition is raised, then it uses *Shrinking* strategies in order to find the smallest assignment that fails the test.

### 2.2.2 Program Visualisation Summary

The program is composed of a mini UI legend that captures the state and a drawing board. Contains functionalities as to the current state of the program and the controls to navigate and operate. Moreover is contains buttons that find solutions or enumerate through a set of solutions.

The controls are summarised as follows. We have two main states, which we denote as node state and operation state. The operation state, determines whether we are adding an edge or a node and the node state determines the type of node that we are adding. We also provide operations to move nodes around or move the whole canvas around. When in edge mode, we can click a node to add an edge between to non-homogeneous nodes, where the order implies the direction of the edge. Moreover, every time you click on a node, you modify its value, so for example for value nodes or for gate nodes you can cycle through its different types by clicking on them. For each gate, we colour-coded its status, so for example if a gate has an invalid arity, then its surrounded by a pink circle or if it has an invalid assignment then its coloured red.

For the main algorithms, we provide two metaheuristic algorithms for finding solutions or approximate solutions, using either evolutionary algorithms or hill climbing search. Moreover for solution enumeration, we are using a modified backtracking algorithm, which upon execution returns a list of solutions. We are limiting the algorithm to at most 20 value nodes in order to keep the computational expenses low. Each time we modify the stucture of the circuit, we reset the solution set. We emphasize that these buttons are disabled if the circuit contains any invalid arities.

### 2.2.3 Solution Selection

For the solution selection we opted with metaheuristic algorithms. Since these problems are conventionally hard to find solutions especially as the number of nodes increase, we focused on the idea of approximation algorithms such in order to find mostly satisfying assingments in large graphs or gadgets. With the help of the `genetic_algorithm`<sup>2</sup> library, we use he *HillClimbing algorithm* and *Evolutionary algorithm* in order to find solutions. We will give a brief explanation on how they work in the upcoming sections but in general we express our graph as a chromosome of *Value* nodes which correspond to nodes in the original graph. Moreover we formulate our problem as series of gates with references to the location of their incident value nodes and we count the of unsatisfying gates. The goal of either algorithm is to minimise the number of errors. Once an assingment is found, we pass the solutions back into our graph and display them.

---

<sup>1</sup><https://github.com/proptest-rs/proptest>

<sup>2</sup>[https://crates.io/crates/genetic\\_algorithm](https://crates.io/crates/genetic_algorithm)

---

## Hill Climbing Algorithm

The idea of hill climbing algorithms is to start by some base solution and repeatedly improve the solution. We allow for 2 strategies, one of them strictly always chooses the best neighbour whereas one allows for stochastic choice. Moreover we allow for additional option such as multiple re-runs in order to increase the odds of finding a successful assignment.

Overall this algorithm, although it runs the fastest out of all the others, it may significantly struggle against large graphs or instances that multiple changes need to occur in order to reach an optimal point. Future ideas can adopt *Simulating annealing* which is a metaheuristic algorithm, where at the start of our run, we allow for a greater variance in our choices and as we progress, we become stricter and stricter with the next neighbour choice.

## Evolutionary Algorithm

Evolutionary algorithms as the names suggests are based of the idea of evolution. A simulation step of an evolutionary algorithm can be summarised in figure .., where we essentially have 5 core elements:

1. **Chromosomes:** Solution assignments of our *PureCircuit* instance
2. **Fitness step:** Evaluation step. We measure how good the assignment is by counting the number of errors it has.
3. **Selection step:** Given a selection strategy, select suitable candidates in order to create the population for the next step. In literature, there are many types that one can use such as *Roulette based* selection, or *Tournament based* selection. We leave several choices available for the user to choose.
4. **Crossover step:** Given two chromosomes from the **Selection step**, combine the genes from each parent, in order to create a new instance.
5. **Mutation step:** Given the offspring of the **Crossover step**, stochastically modify each gene in order to introduce diversity.

The repetition of the steps eventually converges to an assignment that is satisfying. The benefits of these methods over our own personal experimentations is that it seems to scale significantly well. As we aim for the tool to be usable for many situations, we allow for the user to tune the parameters and strategies to their needs in order to adapt to each solution. Moreover, we use speciation techniques were separate our chromosomes to different species in order to maintain diversity and increase the odds of finding a solution.

### 2.2.4 Solution Enumeration

To enumerate through solutions, we employed a traditional backtracking algorithm. We split our algorithm to two main procedures: **Backtrack Initialisation** and **Backtrack Solution Enumerator**.

---

#### [h!] Algorithm 2.2.1 PureCircuit Backtracking Algorithm Enumerator

---

```
function BACKTRACKINITIATION(pc)
   $V \leftarrow PCExtract(pc)$ 
   $S \leftarrow Map(V, KleeneSet :: all()) \triangleright$  Create a state vector that contains the KleeneSet data-structure.
   $Q \leftarrow PRIORITYQUEUEINIT(())$ 
  for  $el \in V$  do
```

---

```

    |   PUSHPQ(Q, el.index, KEYSTRUCT(3, el))
    |   return S, Q
function BACKTRACKPURECIRCUIT(pc :PureCircuit)
    S, Q ← BACKTRACKINSTANTIATION(pc)
    A ← ARR(|V|, ∅)
    r ← BACKTRACKITERATION(pc, A, S, Q)
    |   return r
function BACKTRACKITERATION(pc, A, S, Q)
    if IsEMPTY(Q) then
    |   return [A]
    m ← EXTRACTMIN(Q)
    r ← []
    for v ∈ S[m] do
        S' ← CLONE(S)
        Q' ← CLONE(Q)
        result ← UNITPROPAGATE(pc, m, v, A, S', Q')
        if result = ⊥ then
        |   Continue
        r' ← BACKTRACKITERATION(pc, A, S', Q')
        |   r ← CONCATARRAY(r, r')
    |   return r
function UNITPROPAGATION(pc, m, v, A, S, Q)
    ASSIGNNODE
    PROPAGATENODE
    while Q.is_empty() ∧ Q.peek().key.value_len = 1 do
        ind ← POP(Q)
        v ← POP(S[ind])
        ASSIGNNODE
        |   PROPAGATENODE
    if Q.is_empty() then
    |   return ⊤
    if Q.peek().key.value_len = 0 then
    |   return ⊥
    |   return ⊤
function PROPAGATENODE(pc, m, A, S, Q)
    for g ∈ GETALLGATENEIGHBOURS(pc, m) do
        I ← GETVALUESETS(Input)
        O ← GETVALUESETS(Output)
        (I', O') ← EVALUATESETS(g, I, O)
        PROPAGATESETVALUES(Q, S, I', I)
        PROPAGATESETVALUES(Q, S, O', O)

```

---

**Backtrack Initialisation** We initialise two main structures, a priority queue  $Q$ , and a solution map  $S$  for each node.  $S$  is the a set of possible values a node can take. The priority queue will essentially dictate which nodes to select and evaluate. We want to reduce the recursive calls as much as possible and eliminate any partial assignments that lead to unsatisfying solutions. stores the node references of the value nodes and uses as keys the structure  $\text{KeyStruct}(e)$ . The  $\text{KeyStruct}$  is a structrue that orders elements lexicographically based on the following order:

1. **In descending:** The number of remaining elements
2. **In Ascending:** Whether the node has an *in-degree* of 0

### 3. In Ascending: The total number of neighbours

We argue that the above strategy minimises the recursive calls breadth. Moreover, nodes with *in-degree* of 0 as for each value of their assignment, a solution should exist. Lastly, prioritising nodes with the highest neighbour count implies that we propagate changes to many gates and therefore prioritise the *Unit Application* procedure.

**Unit Propagation operation** The algorithm essentially, assigns the selected value to a node by modifying to the solution set and the value set. Then for each gate that is incident to the node apply the *Set simplification* operation. While the top element of the queue has one possible solution, extract the solution and repeat the *unit value propagation operation* until the queue is empty or the cardinality of the value set at the head of the priority queue is different than 1. If it's zero then current assingment is not valid and we return false, otherwise we return true.

**Set simplification operation** Given a gate, we have a set of possible values for each of our nodes, as determined by our value set. The idea is that we want to reduce the set possible values by following the rules of the gate. For example, we an **AND** gate with triplet of values  $(\{0, \perp, 1\}, \{0, \perp, 1\}, \{0, \perp, 1\})$ . If the output value is assigned the value 1, then we know that the inputs can only take values (1,1) and therefore we restrict the values of the input sets to  $\{1\}, \{1\}$ . If the current solution set of a node is  $v_t$ , then its updated set is  $v_{t+1} \leftarrow v_t \cap \omega$ , where  $\omega$  are the set of values denoted by the simplification operation. We provide the rules of **AND** and **OR** work and in the appendix we provide the full table. If a value is not assigned, we denote it as  $\cdot$ .

| AND input                                                    | Output sets                                                                                  |
|--------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| $\forall v \in \mathbb{T} : (\cdot, \cdot, v)$               | $(\mathbb{T}_{\geq v}, \mathbb{T}_{\geq v}, \{v\})$                                          |
| $\forall v \in \mathbb{T} : (\cdot, v, \cdot)$               | $(\mathbb{T}, \{v\}, \mathbb{T}_{\leq v})$                                                   |
| $\forall a, b \in \mathbb{T} : a < b \implies (\cdot, a, b)$ | Invalid combination                                                                          |
| $\forall v \in \mathbb{T} : (\cdot, v, v)$                   | $(\mathbb{T}_{\geq v}, \{v\}, \{v\})$                                                        |
| $\forall a, b \in \mathbb{T} : a > b \implies (\cdot, a, b)$ | $(\{b\}, \{a\}, \{b\})$                                                                      |
| $\forall a, b \in \mathbb{T} : (a, b, \cdot)$                | $(\{a\}, \{b\}, \{\min(a, b)\})$                                                             |
| $\forall a, b, c \in \mathbb{T} : (a, b, c)$                 | $(\{a\}, \{b\}, \{c\})$ , only if the triplet $(a, b, c)$ is a correct <b>AND</b> assignment |

Table 2.2.1: **AND** value propagation rules

| OR input                                           | Output sets                                                                                 |
|----------------------------------------------------|---------------------------------------------------------------------------------------------|
| $\forall v \in \mathbb{T} : (\cdot, \cdot, v)$     | $(\mathbb{T}_{\leq v}, \mathbb{T}_{\leq v}, \{v\})$                                         |
| $\forall v \in \mathbb{T} : (\cdot, v, \cdot)$     | $(\mathbb{T}, \{v\}, \mathbb{T}_{\geq v})$                                                  |
| $\forall a, b \in \mathbb{T} : a < b(\cdot, a, b)$ | $(\{b\}, \{a\}, \{b\})$                                                                     |
| $\forall v \in \mathbb{T} : (\cdot, v, v)$         | $(\mathbb{T}_{\leq v}, \{v\}, \{v\})$                                                       |
| $\forall a, b \in \mathbb{T} : a > b(\cdot, a, b)$ | Invalid combination                                                                         |
| $\forall a, b \in \mathbb{T} : a > b(a, b, \cdot)$ | $(\{a\}, \{b\}, \{\max(a, b)\})$                                                            |
| $\forall a, b, c \in \mathbb{T} : (a, b, c)$       | $(\{a\}, \{b\}, \{c\})$ , only if the triplet $(a, b, c)$ is a correct <b>OR</b> assignment |

Table 2.2.2: **OR** value propagation rules

**Backtrack Operation** The *backtrack* operation is summarised in 3 key steps. If the priority queue is empty then we found a satisfying assignment. Otherwise, pick a node from the priority queue. For each value in its set, create a copy of the current queue, current solution assingment and value set state and apply the *Unit value propagation* operation. After that create a recursive call apply the *backtrack opertation* again until the current recursion chain terminates with a solution or rejects. At the tree of every execution call, merge the solution assignments in each level and return the sets.

---

## Correctness and Running Time analysis

For the running time analysis, we can observe that each time we assign a node, the unit propagation operation, restricts the solution set of the value nodes that are incident by one gate. Even in the worst case scenario where no reduction occurs, we can clearly see that each time, our priority queue will select a node and will recurse through each of each possible values, creating a call tree with depth of at most  $n$ . Therefore we can argue even if the running time of the algorithm is  $O(3^n)$ , we can observe that the unit propagation and set reduction methods reduce the width and depth greatly.

As for correctness, the algorithm essentially picks nodes in a certain order and selects a value that we believe that will give a satisfying assignment. We only have to look at what happens at the *unit propagation* stage. We can observe that in each stage, we are essentially limiting the set of solutions to only the ones that will give a satisfying assignment. If the value set of a node is zero, it means that no assignment will lead to a solution and therefore we reject the current path. If no nodes are left in the queue, that means that all value nodes are assigned and therefore we can return the current assignment. This implies that our resulting array of assignments contains only valid solutions. A

### 2.2.5 Future Extensions

#### Algorithmic Extensions

Although the aforementioned algorithms work as expected, there are several improvements that can be made. For all three of the algorithmic methods we mentioned, the first and most important extension is to treat weakly directly connected components individually. Trivially an assignment in one weakly connected component should not affect the solution set of the other one. We can therefore reduce the solution spaces of our problems by finding the solution sets in each graph separately. Moreover for the *backtracking* algorithm, we can extend the above idea by considering the cartesian product of the solution sets, meaning: let  $G_i$  denote a (weakly) connected component of  $G$  and  $\text{sol}(\cdot)$  denotes its solution set. Its trivial to observe that

$$\text{sol}(G) = \bigtimes_{k \in [k]} \text{sol}(G_k)$$

We can also optimise the Backtracking algorithm even further. First one can observe that if our graph was acyclic, then the permutation of the nodes with no *in-degree* solutions essentially dictate the assignments of the downstream nodes (excluding nodes incident to purify gates). With this in mind, we can apply the following sequence of steps to make the backtracking algorithm simpler:

1. Identify the strongly connected components of the circuit, using polynomial time algorithms such as Kosaraju's Algorithm or Gabow's Algorithm.
2. Treating SCCs as an individual node, identify nodes by their distances from base nodes (nodes with no incoming edges)
3. Extend the backtracking algorithm so instead of the using the is start label, use the distance from base instead.

We argue that the above heuristic may result in improved average times but of course further research can be done in order to determine more suitable selection heuristics. Lastly an additional heuristic that we failed to consider is using the notion of alternation or varying paths as highlighted in ... Essentially by continuously "propagating" the gate values, one may be able to identify nodes

---

that tent to alternate. This method was used to identify nodes that should be assigned with the  $\perp$  value. It has to be noted that it may not be entirely clear how such construction could be applied as the purify gate has a non-deterministic output, meaning for the  $\perp$  value, the outputs could be multiple. This is the reason why we also don't necessarily propagate solution sets and instead only values as it may lead to infinite loops.



# Bibliography

- [1] S. Arora and B. Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, Cambridge, 2009.
- [2] M. Black. *Critical Thinking: An Introduction to Logic and Scientific Method*. Prentice-Hall Philosophy Series. Prentice-Hall, Incorporated, 1946.
- [3] K. Bronisław. Un theoreme sur les fonctions d'ensembles. 6:133–134, 1928.
- [4] J. Bund, C. Lenzen, and M. Medina. Small Hazard-Free Transducers. *IEEE Transactions on Computers*, 74(5):1549–1564, May 2025.
- [5] S. H. Chan and I. Pak. Computational complexity of counting coincidences. *Theoretical Computer Science*, 1015:114776, Nov. 2024.
- [6] A. K. Chandra, D. C. Kozen, and L. J. Stockmeyer. Alternation. *Journal of the ACM (JACM)*, 28(1):114–133, 1981.
- [7] X. Chen and X. Deng. On the complexity of 2D discrete fixed point problem. *Theoretical Computer Science*, 410(44):4448–4456, Oct. 2009.
- [8] X. Chen, X. Deng, and S.-H. Teng. Settling the complexity of computing two-player Nash equilibria. *J. ACM*, 56(3):14:1–14:57, May 2009.
- [9] K. Claessen and J. Hughes. QuickCheck: A lightweight tool for random testing of Haskell programs. *SIGPLAN Not.*, 35(9):268–279, Sept. 2000.
- [10] S. A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, STOC '71, pages 151–158, New York, NY, USA, May 1971. Association for Computing Machinery.
- [11] C. Daskalakis, P. Goldberg, and C. Papadimitriou. *The Complexity of Computing a Nash Equilibrium*, volume 39. Association for Computing Machinery, Jan. 2006.
- [12] C. Daskalakis, S. Skoulakis, and M. Zampetakis. The Complexity of Constrained Min-Max Optimization, Sept. 2020.
- [13] C. Daskalakis, S. Skoulakis, and M. Zampetakis. The complexity of constrained min-max optimization. In *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing*, STOC 2021, pages 1466–1478, New York, NY, USA, June 2021. Association for Computing Machinery.
- [14] A. Deligkas, J. Fearnley, A. Hollender, and T. Melissourgos. Constant inapproximability for PPA. In *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing*, STOC 2022, pages 1010–1023, New York, NY, USA, June 2022. Association for Computing Machinery.

- 
- [15] A. Deligkas, J. Fearnley, A. Hollender, and T. Melissourgos. Pure-Circuit: Tight Inapproximability for PPAD. *J. ACM*, 71(5):31:1–31:48, Oct. 2024.
- [16] E. B. Eichelberger. Hazard Detection in Combinational and Sequential Switching Circuits. *IBM Journal of Research and Development*, 9(2):90–99, Mar. 1965.
- [17] J. Fearnley, D. Pálvölgyi, and R. Savani. A Faster Algorithm for Finding Tarski Fixed Points. *ACM Trans. Algorithms*, 18(3):23:1–23:23, Oct. 2022.
- [18] S. Friedrichs, M. Függer, and C. Lenzen. Metastability-Containing Circuits. *IEEE Transactions on Computers*, 67(8):1167–1183, Aug. 2018.
- [19] A. Hollender and P. Goldberg. The Complexity of Multi-source Variants of the End-of-Line Problem, and the Concise Mutilated Chessboard, June 2018.
- [20] C. Ikenmeyer, B. Komarath, C. Lenzen, V. Lysikov, A. Mokhov, and K. Sreenivasaiah. On the complexity of hazard-free circuits. *Journal of the ACM*, 66(4):1–20, Aug. 2019.
- [21] C. Ikenmeyer, B. Komarath, and N. Saurabh. Karchmer-Wigderson Games for Hazard-free Computation, Nov. 2022.
- [22] C. Ikenmeyer, K. D. Mulmuley, and M. Walter. On vanishing of Kronecker coefficients. *computational complexity*, 26(4):949–992, Dec. 2017.
- [23] C. Ikenmeyer and I. Pak. What is in #P and what is not? In *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*, pages 860–871, Oct. 2022.
- [24] C. Ikenmeyer, I. Pak, and G. Panova. Positivity of the Symmetric Group Characters Is as Hard as the Polynomial Time Hierarchy. *International Mathematics Research Notices*, 2024(10):8442–8458, May 2024.
- [25] D. S. Johnson, C. H. Papadimitriou, and M. Yannakakis. How easy is local search? *Journal of Computer and System Sciences*, 37(1):79–100, Aug. 1988.
- [26] S. Kleene and M. Beeson. *Introduction to Metamathematics*. Ishi Press International, 2009.
- [27] D. Kozen. *Theory of Computation*. Texts in Computer Science. Springer London, 2006.
- [28] R. H. Makar. On the Analysis of the Kronecker Product of Irreducible Representations of the Symmetric Group. *Proceedings of the Edinburgh Mathematical Society*, 8(3):133–137, Dec. 1949.
- [29] M. Mukaidono. On the B-ternary logic function. *Trans. IECE*, 55:355–362, Jan. 1972.
- [30] I. Pak. What is a combinatorial interpretation?, Sept. 2022.
- [31] C. H. Papadimitriou. On the complexity of the parity argument and other inefficient proofs of existence. *Journal of Computer and System Sciences*, 48(3):498–532, June 1994.
- [32] A. Rubinfeld. Inapproximability of Nash Equilibrium. In *Proceedings of the Forty-Seventh Annual ACM Symposium on Theory of Computing, STOC '15*, pages 409–418, New York, NY, USA, June 2015. Association for Computing Machinery.
- [33] L. G. Valiant. The complexity of computing the permanent. *Theoretical Computer Science*, 8(2):189–201, Jan. 1979.

# Appendices

## .1 The Flux Sketch

